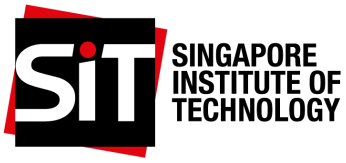


INF1008 Assignment 1 Report

Team ID: P4-07

Name	Student ID	Email
Ee Chew Fong, Olivia	2102743	2102743@sit.singaporetech.edu.sg
Toh Wen Qian	2503151	2503151@sit.singaporetech.edu.sg
Ziven Wang Qian Cheng	2501236	2501236@sit.singaporetech.edu.sg
Yap Yi Hui Wynn	2501386	2501386@sit.singaporetech.edu.sg
Timothy Chia Kai Lun	2501530	2501530@sit.singaporetech.edu.sg

Course: INF1008 – Data Structures and Algorithms

Institution: Singapore Institute of Technology

Date of Submission: 01/02/2026

Contents

Name: Ee Chew Fong, Olivia
Student ID: 2102743

1 Question 1

Write a singly linked list ADT implementation that achieves O(1) time for insertion and removals at any location of the linked list. Your implementation should also provide for a O(1) time get(i) method where i is the position of the item in the list. The source data must be stored as a singly linked list, but you can make use of any additional data structure to facilitate the operations. Explain and justify why your implementation has achieved the stated time complexity requirements.

You are allowed to use any AI tools to assist in your research and development. Clearly document all your research with detail screenshot of the content. Discuss in detail the justification of your derived solution with clear analysis.

1.1 Requirements/Specifications

This implementation is a hybrid architecture — a Singly Linked List Abstract Data Type (ADT) that stores the source data in a singly linked list, where each element is represented by a `Node` containing a data field and a next pointer(GeeksforGeeks, 2026; WsCube Tech, 2025). To support constant-time access by position, an auxiliary indexing structure (`idx`) is maintained such that `idx[i]` holds a direct reference to the i-th node in the linked list. The linked list remains the primary storage of the data, while the auxiliary structure is only to accelerate access to node locations.

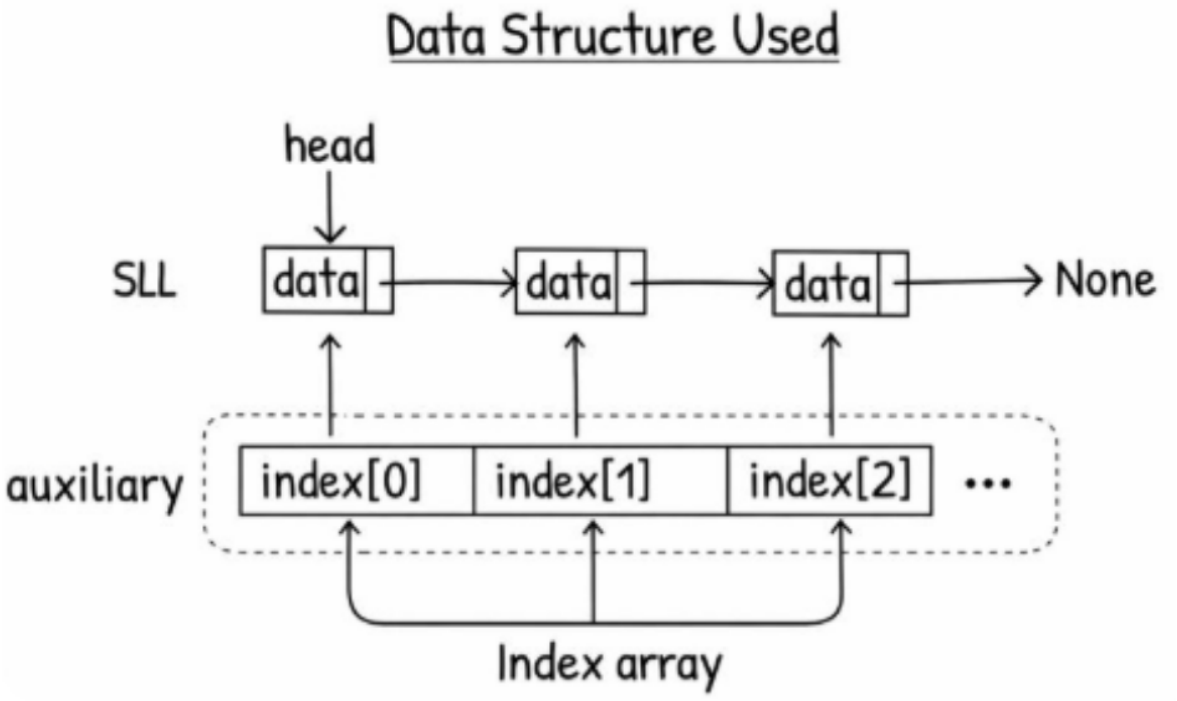


Figure 1: Data Structure

1.1.1 Singly Linked List (SLL)

A linear data structure where each node contains data and a reference to the next node, starting from `head` and terminating at `None`(GeeksforGeeks, 2026; WsCube Tech, 2025).

1.1.2 Auxiliary Indexing Structure (`Idx`)

A Python list that stores direct references to nodes in the singly linked list, where `idx[i]` points to the node at logical position i (Rowell, n.d.).

1.2 User Guide

This program is an interactive console-based application that demonstrates the use of a singly linked list supporting insertion, removal, and display operations. The application allows users to manipulate the list dynamically through a menu-driven interface.

You will need Python version 3 as well as a command-line interface or an IDE that supports Python programs.

1.2.1 Running the Program

First, change the working directory to `Q1/`

```
cd ./Q1/
```

To run the program, access the file using Python.

```
python question_01.py
```

You will be prompted to populate the initial elements in the list and then you will be greeted with a menu interface with the following menu options:

1. Get element
2. Insert element
3. Remove element
4. Display list
5. Exit

You may then enter a number between 1 to 5 based on what you plan to do with the program.

1.2.2 List Population

Users can input multiple elements separated by a comma. Elements will be displayed in the way you enter them from left to right. For example, `a,b,c` will display `[0]a -> [1]b -> [2]c`. You can also press enter to start with an empty list.

1.2.3 Get Operation

The get operation allows you to get an element by index position. The index will always start from 0.

Procedure:

1. Select option 1 from the menu.
2. The program shows you valid positions and asks for input
3. The program displays the result of the input

1.2.4 Insert Operation

The insert operation allows you to add a new element at a specified position in the linked list. **Indices always start from 0 (default empty list); you will have to populate the list first. Insertions will start at the head of the list at index 0.**

Procedure:

1. Select option 2 from the menu.
2. The program displays the current list, its size, and the valid range of insertion positions.
3. Enter the desired position and the data to be inserted.
4. The element is inserted, and the updated list is displayed.

1.2.5 Remove Operation

The remove operation deletes an element from a specified position in the linked list.

Procedure:

1. Select option 3 from the menu.
2. The program displays the current list and valid positions for removal.
3. The user enters the position of the element to remove.
4. The element is removed, and the updated list is displayed.

If the list is empty or an invalid position is entered, an error message is shown.

1.2.6 Display Operation

The display operation shows the current contents of the linked list:

- Select option 4 from the menu.
- Each element is displayed together with its index position.
- If the list is empty, the program indicates that no elements are present.

1.2.7 Exit Operation

Selecting option 5 terminates the program.

1.3 Design and Analysis

The program implements an interactive, menu-driven application that allows users to create and manipulate a singly linked list as the primary data structure. Each element in the list is stored as a node containing user-provided data and a reference to the next node. To support constant-time positional access, an auxiliary index array is maintained, where each index maps directly to a node handle (a direct reference to a node) in the linked list.

The application supports insertion and removal of elements at specified positions using zero-based indexing. By retrieving node handles through the auxiliary index array, insertion and removal operations are performed through pointer manipulation without requiring traversal of the linked list, achieving $O(1)$ time complexity for these operations *once the location/node is known*(GeeksforGeeks, 2026; Rowell, n.d.). The `get(i)` operation similarly retrieves values in $O(1)$ time by directly accessing the corresponding node reference stored in the index array(Rowell, n.d.).

The list is initially empty, and all insert and remove operations update both the singly linked list and the auxiliary index array immediately to preserve positional consistency. The program allows data values of any user-entered type, including alphabetic and alphanumeric strings. Invalid position inputs or non-numeric entries are handled gracefully through informative error messages without terminating execution. Output is presented in a clear, linear format, displaying each element alongside its corresponding index, or an explicit message indicating when the list is empty.

1.3.1 Supported Operations

This data structure supports indexed access, insertion, and removal operations. All positions are 0-based, meaning the head of the list is at position 0.

`get(i)`

The `get(i)` operation returns the data stored at index i. The implementation achieves this by performing direct indexing into the auxiliary array `idx`, which stores references to the corresponding linked list nodes. Since array indexing is constant time, accessing `idx[i]` and returning its data field both take $O(1)$ time(Rowell, n.d.).

Pseudocode

```
function get(i):
    // Check if index is out of bounds
    if i < 0 or i >= length of index array:
        raise IndexError

    // Retrieve node from index array
    node = index[i]

    // Return the data stored in the node
    return node.data
```

`insert(i, data)`

The `insert(i, data)` operation inserts a new node containing `data` at position i.

- If $i = 0$, the new node is inserted at the head of the linked list. The head pointer is updated, and the new node reference is inserted at the beginning of `idx`.
- Otherwise, the node immediately preceding the insertion point is obtained in constant time using `get_node(i-1)`. A new node is then linked into the list by updating a constant number of pointers(GeeksforGeeks, 2026; Tulusibrahim, 2023).
- After insertion, the auxiliary index list `idx` is updated to preserve positional consistency.

Pseudocode

```
function insert(i, data):
    // Create new node with the given data
    new_node = Node(data)

    // Insert new_node after node i in the linked list
    new_node.next = i.next
    i.next = new_node

    // Update tail pointer if inserting after current tail
    if i is tail:
        tail = new_node

    // Insert new_node into index array at the correct position
    insert new_node into index array at the appropriate index
```

`remove(i)`

The `remove(i)` operation deletes the node at position i and returns its stored data.

- If $i = 0$, the head node is removed by updating the head pointer to the next node.
- Otherwise, the node preceding the target is retrieved using `get_node(i-1)`, and its next pointer is updated to bypass the removed node(GeeksforGeeks, 2026; Tulusibrahim, 2023).
- Following removal, the corresponding entry is deleted from `idx`.

Pseudocode

```
function remove(i):
    // Check if index is out of bounds
    if i < 0 or i >= length of index array:
        raise IndexError

    // Special case: removing the head node
    if i == 0:
        return remove_at_head()

    // Get the node before the one to remove
    prev = get_node(i - 1)

    // Remove the node after prev
    removed_data = remove_after(prev)

    // Update index array to maintain consistency
    index.pop(i)

    return removed_data
```

1.3.2 Time Complexity Justification

Operation	What the implementation does	Time	Why
`get(i)`	Uses the auxiliary list to directly index the node: `idx[i]`, then returns `idx[i].data`.	$O(1)$	Array/list indexing by position is constant time; returning a field is also constant time. No traversal or shifting occurs.
Insert (pointer updates only)	After the correct spot is known: `new_node.next = prev.next` then `prev.next = new_node`.	$O(1)$	A fixed number of pointer rewires in a singly linked list, independent of list size.
Remove (pointer updates only)	After the previous node is known: `prev.next = removed_node.next`.	$O(1)$	One pointer change, independent of list size.
`insert(i)` with index maintenance	Insert into `idx`: `idx.insert(i, node)` causes elements at positions $i..n-1$ to shift right.	$O(n)$ (worst case)	While node access and pointer updates are $O(1)$, keeping `idx` in positional order requires shifting up to n references (e.g., insert near head).
`remove(i)` with index maintenance	Remove from `idx`: `idx.pop(i)` causes elements at positions $i+1..n-1$ to shift left.	$O(n)$ (worst case)	Maintaining positional consistency after removal shifts up to n references.

Table 1: Time Complexity Justification

Trade-off between access and modification

The program achieves $O(1)$ random access through the auxiliary index array, which is a significant improvement over the $O(n)$ traversal required in standard linked lists. However, this benefit comes at the cost of $O(n)$ insertions and deletions due to the need to maintain the index array's positional consistency.

Index maintenance

The `insert(i)` and `remove(i)` operations involve `idx.insert()` and `idx.pop()` on a Python list. These operations require shifting all subsequent elements to maintain correct positional mapping. For example:

- Inserting at position 0 requires shifting all n elements rightward.
- Removing from position 0 requires shifting all $n - 1$ remaining elements leftward.
- Operations near the end of the list shift fewer elements but still have $O(n)$ worst-case complexity.

Comparison with Standard Data Structures

A standard linked list has $O(n)$ access but $O(1)$ insertion/removal when you have a node reference. A dynamic array has $O(1)$ random access but $O(n)$ insertion/removal due to shifting elements. This implementation achieves $O(1)$ random access by adding an auxiliary index array, but inherits $O(n)$ modification complexity from the dynamic array, while also adding memory overhead for storing both nodes and index references.

1.3.3 Space Complexity Justification

The auxiliary index array requires $O(n)$ additional space to store references to all n nodes, doubling the space overhead compared to a standard linked list:

$$O(n) + O(n) = O(n)$$

1.4 Limitations of Implementation

The primary limitation of this implementation is that while constant-time access to elements is achieved through an auxiliary indexing structure, insertion and removal operations by index incur a worst-case time complexity of $O(n)$. This is due to the need to maintain positional consistency in the auxiliary index structure, which requires shifting references whenever an element is inserted or removed at the front or middle of the list. As a result, the overall performance of index-based updates degrades as the list size increases.

Additionally, the use of an auxiliary index structure increases the space complexity to $O(n)$, as each node in the singly linked list must also be referenced in the index. While this trade-off enables faster access operations, it requires additional memory compared to a standard singly linked list implementation.

Finally, the interpretation of "location" in this implementation is limited to index-based positions. Insertion and removal operations are constant-time only once the target node location is identified; however, identifying and maintaining locations strictly by index introduces overhead. This is the limitation of an index-based linked list.

1.5 Test Cases

ID	Test Case	Input	Expected Output	Valid?
TC-01	Populate elements into list	`a,b,c`	List based on the order the inputs are written is displayed. (`[0]a -> [1]b -> [2]c`), initial size 3.	Valid
TC-02	Get element at index 2	Menu option: 1; Position: 0; Data: 1	Element at position 2: C	Valid
TC-03	Insert element at head position	Menu option: 2; Position: 0; Data: `1`	Element `1` inserted at position 0; list shows `[0]1` at head; size increases by 1.	Valid
TC-04	Insert element at position 1	Menu option: 2; Position: 1; Data: `2`	Element `2` inserted at position 1; list shows `[1]2`; size increases by 1.	Valid
TC-05	Remove element at tail	Menu option: 3; Position: 4	Element at tail removed; size decreases by 1.	Valid
TC-06	Remove element at position 1	Menu option: 3; Position: 1	Element at position 1 removed; size decreases by 1.	Valid
TC-07	Display current list	Menu option: 4	Displays current list and size.	Valid
TC-08	Exit program	Menu option: 5	Exits program with message.	Valid

Table 2: Test case results

1.6 Listings

```
""" ./Q1/question_01.py """

class Node:
    def __init__(self, data, nxt=None):
        self.data = data
        self.next = nxt


class O1LinkedList:

    def __init__(self):
        self.head = None
        self.idx = [] # Auxiliary index for O(1) access

    @property
    def size(self):
        return len(self.idx)

    # O(1) access
    def get(self, position):
        if position < 0 or position >= self.size:
            raise IndexError("Invalid position")
        return self.idx[position].data

    def get_node(self, position):
        if position < 0 or position >= self.size:
            raise IndexError("Invalid position")
        return self.idx[position]

    # O(1) updates given a handle
    def insert_at_head(self, data):
        new_node = Node(data, self.head)
        self.head = new_node
        # Keep idx consistent for O(1) access
        self.idx.insert(0, new_node)

    def insert_after(self, node, data):
        new_node = Node(data, node.next)
        node.next = new_node
        return new_node

    def remove_at_head(self):
        if self.head is None:
            raise IndexError("List is empty")
        removed = self.head
        self.head = removed.next
        self.idx.pop(0)
        return removed.data
```

```
    def remove_after(self, node):
        if node.next is None:
            raise IndexError("Nothing to remove at that location")
        removed = node.next
        node.next = removed.next
        return removed.data

    def insert(self, position, data):
        if position < 0 or position > self.size:
            raise IndexError("Invalid position")

        if position == 0:
            self.insert_at_head(data)
            return

        prev = self.get_node(position - 1)
        new_node = self.insert_after(prev, data)

        # Maintain idx so future get_node() works
        self.idx.insert(position, new_node)

    def remove(self, position):

        if position < 0 or position >= self.size:
            raise IndexError("Invalid position")

        if position == 0:
            return self.remove_at_head()

        prev = self.get_node(position - 1)
        removed_data = self.remove_after(prev)

        # Maintain idx so future get_node() works
        self.idx.pop(position)
        return removed_data

    # Display
    def display_with_indices(self):
        if self.size == 0:
            print("List: [Empty]")
            return

        curr = self.head
        out = []
        i = 0
        while curr:
            out.append(f"[{i}]{curr.data}")
            curr = curr.next
            i += 1
        print("List: " + " -> ".join(out))
```

```
def populate_list(ll):
    """Allow user to populate the list with initial data"""
    print("\n" + "=" * 60)
    print("  POPULATE LIST")
    print("=" * 60)
    print("Enter initial elements for the list.")
    print("You can enter multiple elements separated by commas,")
    print("or press Enter to skip and start with an empty list.")
    print("=" * 60)

    user_input = input("\nEnter elements (e.g., A,B,C or 1,2,3): ").strip()

    if not user_input:
        print("\nStarting with an empty list.")
        return

    # Split by comma and strip whitespace
    elements = [elem.strip() for elem in user_input.split(',') if elem.strip()]

    # Insert each element at the end of the list
    for elem in elements:
        ll.insert(ll.size, elem)

    print(f"\n[SUCCESS] Added {len(elements)} element(s) to the list.")
    ll.display_with_indices()
    print(f"Initial size: {ll.size}")
    input("\nPress Enter to continue to main menu...")


def print_menu():
    print("\n" + "=" * 60)
    print("  LINKED LIST - INTERACTIVE TESTER")
    print("=" * 60)
    print("1. Get element")
    print("2. Insert element")
    print("3. Remove element")
    print("4. Display list")
    print("5. Exit")
    print("=" * 60)


def main():
    ll = O1LinkedList()

    print("\n*** Singly Linked List - Interactive Program ***")

    # Populate list first
    populate_list(ll)

    print("\nMenu accepts positions; list updates are shown after each operation.")
```

```
while True:
    print_menu()
    choice = input("\nEnter your choice (1-5): ").strip()

    if choice == "1":
        print("\n" + "-" * 60)
        print("GET OPERATION")
        print("-" * 60)

        if ll.size == 0:
            print("Error: List is empty! Nothing to get.")
            input("\nPress Enter to continue...")
            continue

        ll.display_with_indices()
        print(f"Current size: {ll.size}")
        print(f"Valid positions: 0 to {ll.size - 1}")

        try:
            position = int(input("Enter position to get: "))

            data = ll.get(position)

            print("\n[RESULT]")
            print(f"Element at position {position}: '{data}'")

        except ValueError:
            print("Error: Please enter a valid number for position.")
        except IndexError as e:
            print(f"Error: {e}")

    elif choice == "2":
        print("\n" + "-" * 60)
        print("INSERT OPERATION")
        print("-" * 60)

        ll.display_with_indices()
        print(f"Current size: {ll.size}")
        print(f"Valid positions: 0 to {ll.size}")

        try:
            position = int(input("Enter position to insert: "))
            data = input("Enter data to insert: ").strip()

            ll.insert(position, data)

            print("\n[RESULT]")
            print(f"Inserted '{data}' at position {position}")
            ll.display_with_indices()
            print(f"New size: {ll.size}")
```

```
            except ValueError:
                print("Error: Please enter a valid number for position.")
            except IndexError as e:
                print(f"Error: {e}")

        elif choice == "3":
            print("\n" + "-" * 60)
            print("REMOVE OPERATION")
            print("-" * 60)

            if ll.size == 0:
                print("Error: List is empty! Nothing to remove.")
                input("\nPress Enter to continue...")
                continue

            ll.display_with_indices()
            print(f"Current size: {ll.size}")
            print(f"Valid positions: 0 to {ll.size - 1}")

            try:
                position = int(input("Enter position to remove: "))

                removed_data = ll.remove(position)

                print("\n[RESULT]")
                print(f"Removed '{removed_data}' from position {position}")
                ll.display_with_indices()
                print(f"New size: {ll.size}")

            except ValueError:
                print("Error: Please enter a valid number for position.")
            except IndexError as e:
                print(f"Error: {e}")

        elif choice == "4":
            print("\n" + "-" * 60)
            print("CURRENT LIST")
            print("-" * 60)
            ll.display_with_indices()
            print(f"Size: {ll.size}")

        elif choice == "5":
            print("\nThank you for using the Linked List Tester!")
            print("Exiting program...")
            break

        else:
            print("Error: Invalid choice. Please enter 1-5.")

        input("\nPress Enter to continue...")
```

```
if __name__ == "__main__":
    main()
```

1.7 AI Assistance Documentation

The initial singly linked list implementation was analysed to determine whether it achieved $O(1)$ time complexity for insertion and removal operations. It was discovered that due to the need to shift elements in the auxiliary structure, the implementation contains loops proportional to the size of the list, resulting in $O(n)$ time complexity for both insert and remove operations in the worst case.

does my code achieve an O(1) complexity ?

No — **your code does not achieve O(1) insert/remove at any position.** It's **O(n)** in the worst case (and typically), because you **shift the index_map entries** on every insert/remove.

Where it becomes O(n)

In `insert()`:

python Copy code

```
for i in range(self.size, position, -1):
    self.index_map[i] = self.index_map[i - 1]
```

That loop can run ~`n` times (e.g., insert at position 0), so **O(n)**.

In `remove()`:

python Copy code

```
for i in range(position, self.siz
    self.index_map[i] = self.index_map[i + 1]
```

Ask ChatGPT

Also can run ~`n` times (e.g., remove at position 0), so **O(n)**.

So even though pointer rewiring in the linked list is O(1), your **aux structure maintenance is not.**

What complexities you actually have

- `get(i)` via `index_map[i]` : **O(1)** average (dict lookup)
- `insert(position, data)` : **O(n)** due to shifting
- `remove(position)` : **O(n)** due to shifting

To address this, the recommended approach is to design the data structure as an Abstract Data Type (ADT) consisting of a core singly linked list and an auxiliary structure to enable $O(1)$ access to node locations.

You can **store the data in a singly linked list**, and you **can** make:

- `get(i)` = **O(1)** (worst-case),
- **insert/remove** = **O(1)** *at any location* **only if "location" means "a node you already have a reference to"** (a handle / iterator).

If "location" is strictly "by index position i" **and** you must keep indices 0..n−1 always valid after every insert/remove, then **worst-case O(1) for all three at once is not achievable** with a singly linked list using standard RAM assumptions — because changing the order by index inherently forces **many indices to change** after an insertion/deletion.

So the right way to satisfy what most lecturers actually mean here is:

> Make `get(i)` return a **node reference (handle)** in O(1).
> Then insert/remove at that location using that handle in O(1).

Key API idea (the trick)

Expose operations that use a **location handle**:

- `get_node(i)` returns the node reference in O(1)
- `insert_after(node_ref, data)` is O(1)
- `remove_after(node_ref)` is O(1)
- Special-case: inserting/removing at head is O(1)

The revised implementation achieves $O(1)$ time complexity for the get(i) operation through direct indexing into the auxiliary structure. However, insert and remove operations still have a worst-case time complexity of $O(n)$ due to the need to maintain positional consistency by shifting elements in the auxiliary structure. This highlights the inherent limitation of index-based insertion and removal when positional ordering must be preserved.

ok so only get is O(1) insert and remove will gradually be O(n)

Yes — **that statement is correct**, with one small wording tweak to be precise and exam-safe.

The accurate conclusion (use this)

> **Only `get(i)` is O(1).**
> **Insert and remove operations are O(n) in the worst case**, because maintaining positional indexing requires shifting elements in the auxiliary array.

That's the **correct, honest Big-O analysis** of your code **as written in Python**.

Summary without index

Operation	Time Complexity
`get(i)`	O(n)
`insert(i)`	O(n)
`remove(i)`	O(n)

So **everything becomes O(n)**.

Why this fails the assignment requirement

The question explicitly asks for:

- O(1) `get(i)`
- O(1) insert and remove *at a location*

Without an index:

- You **cannot** get to position `i` in O(1)
- Therefore, the requirement is **not met**

but is it possible for O(1)

Short, honest answer first:

👉 **No — it is NOT possible to achieve true worst-case O(1) insertion and removal *by index* while also having O(1) `get(i)` for a singly linked list**, if indices must remain correct.

However, get, insert/remove at $O(1)$ can be achieved if the assignment does not require the list to be indexed. Where Stable IDss can be used to locate the data. For the purpose of this project and from my interpretation, I will assume that we need it to be indexed, hence, an $O(1)$ complexity for all 3 - get, insert and remove is not possible due to shifting.

Option 2: "Position" = Stable ID ✅

Yes, if "position" means stable identifier (not sequential 0,1,2,3), then the stable ID implementation fulfills this perfectly:

- ✅ O(1) insert - returns an ID
- ✅ O(1) remove - by ID
- ✅ O(1) get(i) - where i is the stable ID

Created a code that allows the users to input elements into the list before displaying in insert, remove and display menu.

based on my code if i want to show O(1) insert and delete should i not ask users to populate the list first?

Yes, exactly! If you want to demonstrate **O(1) insert and delete** (the pure pointer operations), you should:

1. **Have users populate the list first** with some initial data
2. **Then provide options to:**
   - Insert after a specific node (using `insert_after(node, data)`) - O(1)
   - Remove after a specific node (using `remove_after(node)`) - O(1)
   - Insert at head (using `insert_at_head(data)`) - but this is O(n) due to `idx.insert(0, new_node)`
   - Remove at head (using `remove_at_head()`) - but this is O(n) due to `idx.pop(0)`

Name: Toh Wen Qian
Student ID: 2503151

2 Question 2(a)

The complexity of an algorithm can be defined using asymptotic notations. The order of growth for an algorithm can be classified into constant, logarithmic, linear, linearithmic, quadratic, exponential and factorial. Explain in detail the characteristics of each type. Further elaborate linearithmic and exponential with 2 algorithms to demonstrate its behavior. One algorithm showing linearithmic order of growth. The other algorithm shows exponential order of growth.

Note: *This must be a new algorithm which is not in our lectures or tutorials.*

2.1 Introduction

Asymptotic notation is a way to describe how an algorithm behaves when the amount of data it works on becomes larger. Instead of focusing on the exact time an algorithm takes to run, it looks at the pattern of growth, in other words, how the workload increases as the input size increases. This is useful because the actual running time can be affected by the computer or programming language used, but the growth pattern of an algorithm remains the same. Asymptotic notation allows algorithms to be compared in a fair and consistent way.

The main forms of asymptotic notation are Big-O, Big-Ω, and Big-Θ. Big-O describes the upper bound (worst-case growth) and is the most commonly used in practice. Big-Θ is often preferred when possible because it provides a tight bound, precisely describing both the upper and lower growth rates, giving the most accurate representation of an algorithms performance.

Algorithm performance is commonly classified into seven growth categories: constant, logarithmic, linear, linearithmic, quadratic, exponential, and factorial. These categories describe how resource usage scales with input size. In general, algorithms with slower growth rates are more scalable and preferred for handling large problems. Table 3 presents the different growth types, arranged from the slowest to the fastest growth.

Growth Type	Explanation
Constant ($O(1)$)	Work does not depend on input size.
Logarithmic ($O(\log n)$)	Input size is reduced by a constant factor each step.
Linear ($O(n)$)	Work grows proportionally with input size.
Linearithmic ($O(n \log n)$)	Multiple "levels" of processing over all items (e.g., divide-and-conquer).
Quadratic ($O(n^2)$)	Work grows with the number of pairs of items (often nested loops).
Exponential ($O(2^n)$)	Work roughly doubles with each additional input element.
Factorial ($O(n!)$)	Work grows with the number of permutations/orderings.

Table 3: Comparison of Asymptotic Growth Rates

2.1.1 Constant $O(1)$

Growth: Constant time, $O(1)$, means the algorithm always takes the same amount of time regardless of input size. The number of operations is fixed and does not depend on how much data is given.

Key characteristics: No loops over input, direct access to data, and input size does not affect the run time. Most basic operations take a constant amount of time such as arithmetic, assignment, and indexing.

Example code structure:

```
value = cache[key]  # direct lookup
```

Numeric example: Accessing the first element of a list takes roughly the same time whether the list has 5 elements or 5 million.

Example algorithm: Direct lookup of a value in a hash map, such as checking a cache before querying a database.

Practicality: This is the most efficient growth type and is ideal for performance-critical operations where speed is very important, such as caching frequently accessed data or retrieving configuration values.

2.1.2 Logarithmic $O(\log n)$

Growth: Execution time increases very slowly as input size grows. With each main step, the problem size is reduced by a constant factor (usually half), so the number of steps grows proportional to $\log n$.

Key characteristics: The algorithm uses a loop or recursion that progressively narrows the search range, discarding large portions of the input at each step, so it does not process every element and typically relies on sorted or structured data, resulting in only a small number of iterations even for very large inputs.

Example code structure:

```
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
```

Numeric example: Searching through 32,768 items by repeatedly halving the search space requires only about 15 steps, showing that the execution time increases very slowly even as the input size becomes large.

Example algorithm: Binary search on a sorted list or searching for a key in a balanced binary search tree.

Practicality: Logarithmic time is efficient and scales well to large datasets, making it one of the most desirable growth types in practice.

2.1.3 Linear $O(n)$

Growth: The execution time increases in direct proportion to the input size n. If we add more elements to the input, the algorithm does a constant amount of extra work for each new element, so the total running time grows like $O(n)$. This means that when n doubles, the number of operations also roughly doubles.

Key characteristics: Processes each element once using a single pass with a constant-time operation per element.

Example code structure:

```
for item in items:
    process(item)
```

Numeric example: Processing a list of 1,000 items takes 1ms, then processing 2,000 similar items will take about 2ms, and 10,000 items will take about 10ms, assuming its executed in the same environment.

Example algorithm: Finding the maximum value in a list by starting from the first element, tracking the current maximum, and updating it whenever a larger value is seen.

Practicality: Linear time is generally considered acceptable even for fairly large inputs, especially when compared with quadratic or exponential algorithms, and it appears very frequently in realworld programs and dataprocessing tasks.

2.1.4 Linearithmic $O(n \log n)$

Growth: Linearithmic growth occurs when a problem is repeatedly divided into smaller parts, while all items are still processed at each stage. Each division reduces the problem size by half, resulting in a small number of stages. At every stage, the algorithm examines all input elements once. Because the input is processed across several division stages, the total amount of work grows faster than linear time but much slower than quadratic time.

Key characteristics: The algorithm repeatedly splits the input into smaller parts, but still processes all elements at each stage, which causes the running time to grow as $nlogn$.

Example code structure:

```
items.sort()          # O(n log n)
for item in items:    # O(n)
    aggregate(item)
```

Numeric example: If $n = 1{,}000$, the input is divided into about 10 levels, and each level processes all 1,000 elements. If $n = 1{,}000{,}000$, there are about 20 levels, with all elements processed at each level. The total work increases steadily but far less than quadratic growth.

Example algorithm: A custom data aggregation routine that splits a dataset into halves, processes each part independently, and then merges the partial results.

Practicality: Linearithmic time is considered efficient and is often the practical target for complex problems on large datasets, as it scales much better than quadratic algorithms while still being achievable for many realworld divideandconquer designs.

2.1.5 Quadratic $O(n^2)$

Growth: The execution time grows with the square of the input size n. The algorithm performs work for each pair of elements, so the total number of basic operations is on the order of n^2 When the input size doubles, the amount of work increases by roughly four times, and when it triples, the work increases by about nine times.

Key characteristics: A structure with two nested loops where the outer loop runs over all n elements and, for each element, the inner loop also runs over all n elements (or almost all), performing a constanttime operation for each pair.

Example code structure:

```
for i in range(n):
    for j in range(n):
        compare(i, j)
```

Numeric example: If a list of 1,000 items leads to about 1,000,000 pairwise checks, increasing the list to 2,000 items results in roughly 4,000,000 checks. For 10,000 items, the number of checks jumps to around 10,000,000, showing how quickly the cost grows.

Example algorithm: A naive conflict-detection algorithm that compares every pair of bookings to check for overlapping time slots in a scheduling system.

Practicality: Quadratic time may be acceptable for small datasets, but it scales poorly as input size increases. For larger problems, the rapid growth in operations makes these algorithms inefficient compared to linear or linearithmic approaches, which are generally preferred in real-world systems.

2.1.6 Exponential $O(2^n)$

Growth: The execution time increases roughly in proportion to 2^n, where n is the input size. Every time one more element is added, the amount of work almost doubles because the algorithm must consider both possibilities for each element. As a result, even small increases in input size cause a massive increase in computation, making the algorithm impractical very quickly

Key characteristics: The algorithm explores all possible combinations by branching into two choices for each input element, forming a decision tree whose size doubles at every level.

Example code structure:

```
def explore(i):
    if i == n:
        return
    explore(i + 1)
    explore(i + 1)
```

Numeric example: With 5 elements, the algorithm checks 32 possibilities. With 10 elements, this increases to 1,024 possibilities. With 20 elements, it reaches about 1 million possibilities. By the time the input reaches 30 elements, the number of possibilities exceeds 1 billion. This shows how quickly exponential growth becomes unmanageable.

Example algorithm: An algorithm that generates all possible subsets (combinations) of a set of items to test which subsets satisfy a condition, for example checking every possible combination of features to see which set produces a desired outcome.

Practicality: Exponential algorithms are only suitable for very small input sizes. They are often used as brute-force baselines or when no more efficient solution is known. However, they do not scale and are quickly replaced by more efficient approaches whenever possible.

2.1.7 Factorial $O(n!)$

Growth: The execution time grows extremely fast as the input size increases. For an input of size n, the algorithm needs to consider all possible orderings of the elements, which results in $n!$ combinations. Each time one more element is added, the amount of work multiplies, causing the running time to explode very quickly. This growth is even faster than exponential time and becomes impractical even at very small input sizes.

Key characteristics: The algorithm recursively generates every possible ordering by fixing one element at a time and permuting the remaining elements.

Example code structure:

```
def permute(items):
    if not items:
        return
    for i in range(len(items)):
        permute(items[:i] + items[i+1:])
```

Numeric example: When $n = 5$, there are $5! = 120$ possible orderings. When $n = 10$, this increases to $10! = 3,628,800$ orderings. By $n = 15$, the number reaches over 1 trillion, which is far beyond what can be processed in a reasonable amount of time.

Example algorithm: A brute-force algorithm that checks every possible delivery route to find the shortest path that visits each customer once, similar to an exhaustive solution to the travelling salesman problem.

Practicality: In real-world applications, factorial-time algorithms are impractical to use in production because they do not scale. Instead, they are mainly used for small test cases or as reference models to compare against more efficient algorithms. Real systems typically replace them with heuristic or approximation approaches that trade perfect optimality for speed and scalability.

While we have compared the different growth types, it does not fully show how these growth patterns arise in real algorithms. To address this, two algorithms are examined to illustrate linearithmic and exponential growth, as they represent two key and contrasting points in algorithm efficiency.

Linearithmic growth is the efficiency we usually aim for when designing algorithms, because it still scales well even as the input size grows large. On the other hand, exponential growth shows how some algorithm designs quickly become unrealistic, since the amount of work explodes with every additional input. Looking at both growth patterns side by side makes it clear why good algorithm design is so important for scalability and real-world performance.

2.2 Requirements/Specification

This program demonstrates the behaviour of two different algorithmic growth rates using practical scenarios. We assumed that input data follows the specified format. Basic validation is applied to ensure required fields are present and values are of valid types.

1. The first part implements a linearithmic-time algorithm that aggregates e-commerce sales by region using a sort-then-scan approach. The input is a list of orders, where each order contains an order ID, region, and sales amount. The output is a summary showing the total sales amount for each region.

2. The second part implements an exponential-time algorithm that generates all possible combinations of access control permissions. The input is a list of permission names, and the output is a list of all possible permission configurations, where each permission can be either granted or not granted.

2.3 User Guide

Before executing the script, ensure Python 3 is installed on the system. Next, change the working directory to `Q2A/`

```
cd ./Q2A/
```

To run the demonstration script:

```
python question_2a.py
```

To run the test cases

```
python question_2a_test.py
```

The program will automatically:

- Execute test cases for the linearithmic algorithm
- Execute test cases for the exponential algorithm
- Display results and validation outcomes in the terminal

2.4 Design and Analysis

2.4.1 Linearithmic Algorithm $O(n \log n)$

Scenario: Imagine we run an e-commerce store and receive a large number of orders. Each order contains an order ID, a region (e.g., SG, MY, JP), and a sales amount. The goal is to compute the total sales for each region efficiently, even when the number of orders is large.

To achieve this, the algorithm uses a **sort-then-aggregate** strategy. The algorithm consists of three main phases:

Step 1: Input Validation $O(n)$. Before processing the data, the algorithm validates the input to ensure correctness. Each order is checked to confirm that:

- The input is a list of dictionaries
- Each dictionary contains a valid region and amount
- The region is a non-empty string
- The amount is a numeric value

Since each order is checked once, this step takes $O(n)$ time.

Step 2: Sort orders by region using Heap Sort $O(n \log n)$. The goal is to group orders by region (JP MY SG) so that orders from the same region appear together. We use Heap Sort (GeeksforGeeks, 2025), which consists of two main phases:

1. **Build Phase** Convert the array into a Max Heap
2. **Sort Phase** Repeatedly extract the maximum element and restore the heap

Phase 1: Build the Max Heap $O(n)$. For example we have the following order list: `[SG100, MY200, JP150, SG300, MY250, JP175, SG400, MY225]`

To build the Max Heap:

- Start checking from the last parent node in the array, ignoring leaf node such as MY225
- At each position, it make sure the parent is larger than its children

If the parent is smaller than one of its children:

- Swap it with the larger child
- Keep moving downward and checking again
- Stop when the parent is larger than both children

Example of Max Heap

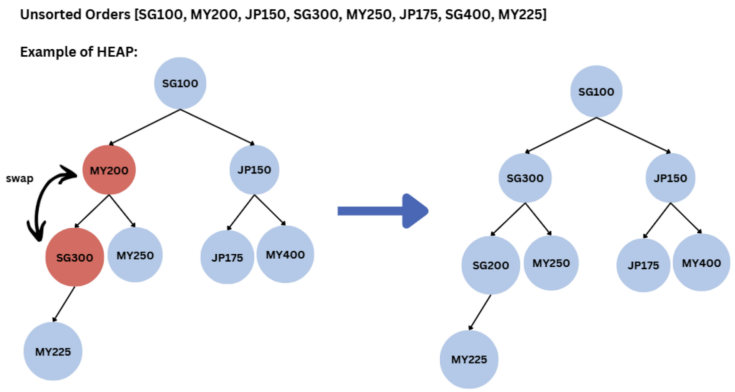


Figure 2: Initial array

If the parent is smaller, it swaps with the larger child and continues checking downward. This checking-and-swapping process continues until all parent nodes satisfy the heap condition. Only the initial and final states are shown for clarity.

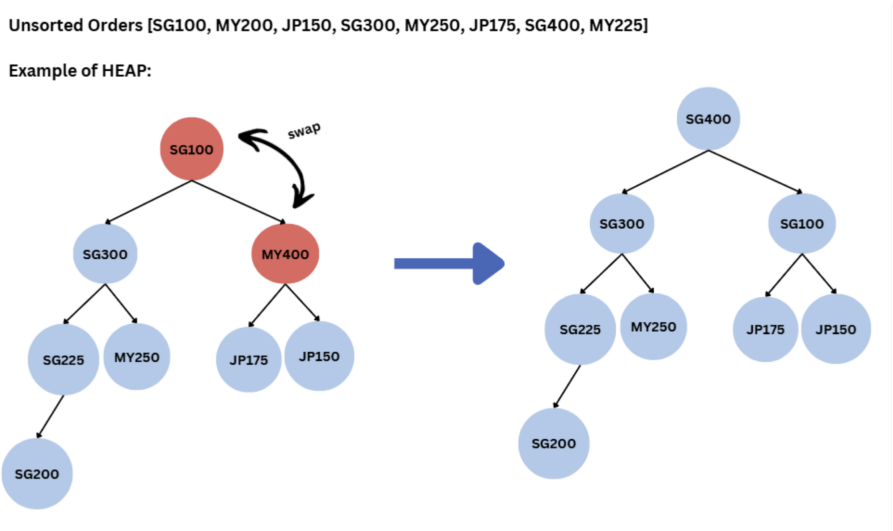


Figure 3: Final heapify step.

After this process, the structure becomes a Max Heap, where the largest element is at the top. This build phase runs in $O(n)$ time.

Phase 2: Sorting by Repeated Extraction $O(n \log n)$**.** For example, using the heap where **SG400** is at the top:

- **SG400** (the largest element) is moved to the end of the list
- It is removed from further consideration and stays in its final position
- The new top element is then adjusted downward so that the next largest value moves back to the to

This process continues until all elements are placed in their correct positions. Since the heap is a tree that grows wider rather than taller, its height increases slowly as the number of elements increases. Adjusting one element can only move it down along the height of the tree, which takes $O(\log n)$ time. Repeating this for n elements results in a total time complexity of $O(n \log n)$.

Step 3: Scan and aggregate totals $O(n)$**.** The sorted list is scanned once from start to end. A dictionary is used to accumulate the total sales for each region. Since each order is visited exactly once, this step takes $O(n)$ time.

Time complexity analysis. The time complexity is analyzed using the RAM model, where each basic operation such as comparisons, assignments, list access, and average-case dictionary lookups takes constant time $O(1)$. Let n be the number of orders in the input list.

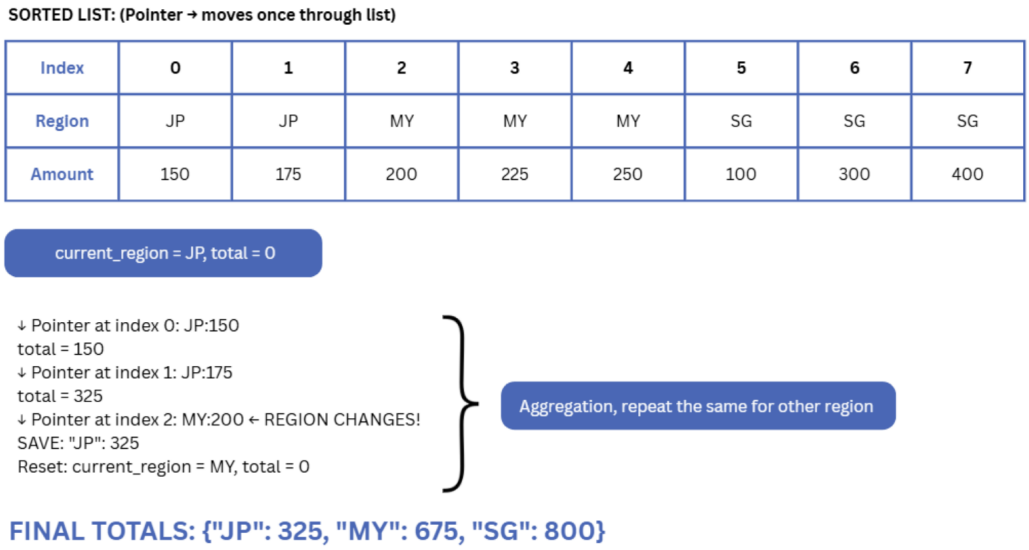


Figure 4: Linear aggregation pass after sorting.

Input Validation - $O(n)$**.** The algorithm first checks if the input is a list of dictionaries, which takes $O(1)$ time. It then loops through all n orders to validate their structure and values. Each order is checked once using constant-time operations, so the total time for input validation is $O(n)$.

Sorting Using Heap Sort $O(n \log n)$**.** A copy of the input list is created to avoid modifying the original data, which takes $O(n)$ time. The list is then sorted by region using Heap Sort. Building the heap takes $O(n)$ time, and extracting elements from the heap is done $n-1$ times, with each extraction taking $O(\log n)$ time. Therefore, the overall time complexity of the sorting step is $O(n \log n)$.

Aggregation Scan $O(n)$**.** After sorting, the algorithm scans the list once to aggregate total sales by region. Each iteration performs constant-time dictionary updates, so this step takes $O(n)$ time. The total time complexity is:

$$O(n) + O(n \log n) + O(n) = O(n \log n)$$

The sorting step dominates, so the algorithm has linearithmic time complexity $O(n \log n)$.

2.4.2 Exponential Algorithm

Scenario:
In this scenario we are working with an access control system. Users or roles can be assigned permissions such as **Read**, **Write**, **Delete**, or **Approve**. Each permission can either be **granted** or **not granted**. For tasks like security auditing or configuration testing, we may want to examine **every possible permission configuration**, especially when the number of permissions is small.

The algorithm uses a recursive backtracking approach. For each permission, we make two choices:

1. Grant the permission
2. Do not grant the permission

The algorithm explores both choices and then moves on to the next permission. This process continues until a decision has been made for every permission, producing one complete permission combination. By repeating this process, the algorithm generates all possible combinations.

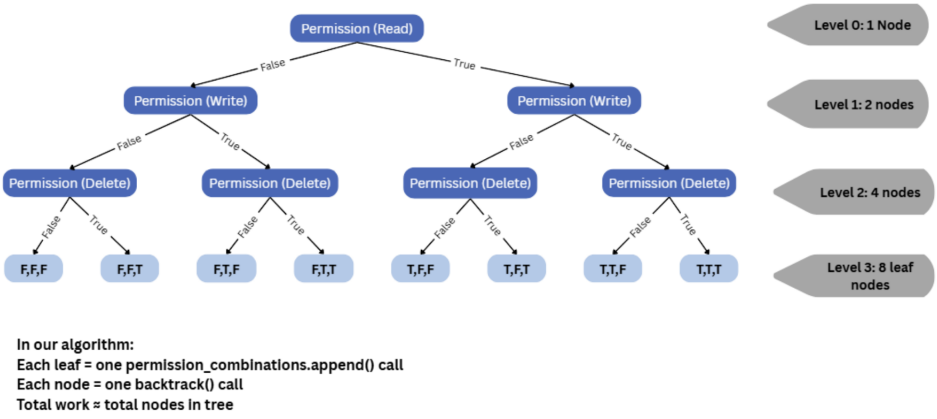


Figure 5: Illustration of exponential growth ($O(2^n)$).

Time complexity analysis.

- If there are n permissions, each permission doubles the number of possible configurations.
- This results in 2^n total combinations.
- The recursion tree grows exponentially, doubling at each level, which leads to an overall time complexity of $O(2^n)$.

This clearly demonstrates how exponential algorithms become impractical very quickly as the input size increases, and why they are typically used only for small inputs or demonstration purposes.

2.5 Limitations

2.5.1 Linearithmic Algorithm

- The same task of computing the total sales per region can be done in $O(n)$ time using a hash-based aggregation without sorting. In this implementation, Heap Sort is used to order the data to demonstrate $O(nlogn)$ time complexity, rather than to achieve the most optimal performance.

- The program assumes a fixed input structure where each order must contain the keys `region` and `amount`. It cannot be reused for different data schemas or multiple grouping keys (for example, grouping by both region and date) without modifying the code.
- If any order contains invalid data (missing keys or incorrect data types), the function raises an exception and stops execution.
- It does not skip invalid records or continue processing the remaining valid orders.
- Sales amounts are handled using standard numeric types (`int` or `float`). It does not handle high-precision financial rounding.

2.5.2 Exponential Algorithm

- This algorithm generates and stores all 2^n possible permission configurations in memory. As n increases, both time and memory usage grow exponentially, making the algorithm impractical for large input sizes.
- Each permission is treated as an independent binary flag. The algorithm does not model real-world permission constraints or dependencies (such as prerequisite permissions).
- The function returns all configurations at once, rather than generating them incrementally. This limits usability in memory-constrained environments.
- The algorithm expects the input to be a non-negative integer representing the number of permissions. It does not support named permissions or more complex permission structures without modification.

2.6 Testing

For both algorithms, the tests were designed to cover:

- Normal cases with typical inputs (multiple regions; $n = 1$ and $n = 3$ permissions).
- Edge cases such as an empty order list and a single order, and $n = 0$ permissions (which should return 1 empty configuration).
- Stress/behaviour cases including a larger order list (20 orders) and multiple regions in random order, to confirm aggregation works correctly after sorting.
- Invalid inputs to verify the functions do not fail silently:
  - Non-numeric amount values should raise a `ValueError`.
  - Missing required keys (e.g., missing `amount`) should raise a `KeyError`.
  - Negative permission counts ($n < 0$) should raise an exception.

All tests were implemented in a separate file `Question2a_Test.py`, which imports `aggregate_sales_by_region` and `generate_permission_combo` from `Question2a.py`, then runs each function using predefined inputs and prints the actual outputs alongside the expected results.

2.6.1 Linearithmic Algorithm

Test ID	Input description	Expected output	Actual result
1	Empty order list	{}	Printed {} – Pass
2	Single order in region SG	{'SG': 120.0}	Matches – Pass
3	Three SG orders (same region)	{'SG': 100.0}	Matches – Pass
4	Orders already sorted by region (JP, MY, SG)	{'JP': 50.0, 'MY': 25.0, 'SG': 75.0}	Matches – Pass
5	Random order, regions SG, MY, JP, US	{'SG': 55.0, 'MY': 25.0, 'JP': 5.0, 'US': 50.0}	Matches – Pass
6	Larger list of 20 mixed orders	Correct totals per region (SG, MY, JP, US)	Matches – Pass
7	Invalid: amount is a string "abc"	Raise ValueError	Exception raised – Pass
8	Invalid: missing amount key	Raise KeyError	Exception raised – Pass

Table 4: Test Cases for Linearithmic Algorithm

2.6.2 Exponential Algorithm Testing Strategy

We tested the exponential algorithm with small permission list sizes (to keep 2^n manageable) and with invalid inputs. Permission list sizes were intentionally kept small to avoid exponential explosion, as the number of generated combinations doubles with each additional permission.

Test ID	Input description	Expected output	Actual result
1	$n = 0$	1 configuration (empty configuration)	Pass
2	$n = 1$	2 configurations	Pass
3	$n = 3$	8 configurations	Pass
4	$n = -1$	Raise error for invalid input	Pass

Table 5: Test Cases for Exponential Algorithm

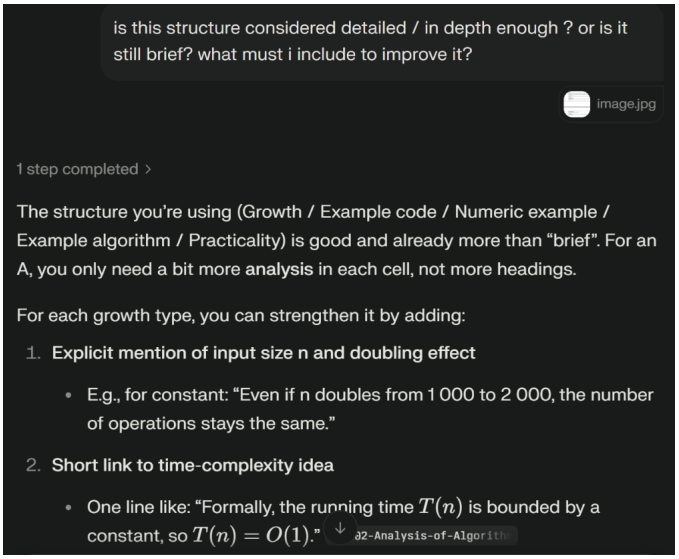
2.7 Listings

```
""" ./Q2A/question_2a.py """

from numbers import Real

# ---------- CODE FOR LINEARITHMIC ALGORITHM O(n log n) DEMO : aggregate_sales_by_region ---
# Function: aggregate_sales_by_region
# Purpose:
#   1) Validate input data
#   2) Sort orders by region using Heap Sort (O(n log n))
#   3) Aggregate total sales per region in one pass (O(n))

def aggregate_sales_by_region(order_list):
    """
    This function demonstrates a linearithmic time algorithm O(n log n).

    Step 1: Sort the orders by region using Heap Sort -> O(n log n)
    Step 2: Traverse the sorted list once to sum sales -> O(n)

    Input validation rules:
    - order_list must be a list
    - each element must be a dictionary
    - each dictionary must contain:
        - 'region': non-empty string
        - 'amount': real number (int or float, not bool)
    """
    # --------------------
    # Input Validation
    # --------------------
    if not isinstance(order_list, list):
        raise TypeError("order_list must be a list of orders (dicts).")

    for i, order in enumerate(order_list):
        # Each order must be a dictionary
        if not isinstance(order, dict):
            raise TypeError(f"Order at index {i} must be a dict.")

        # Required keys check
        if "region" not in order:
            raise KeyError(f"Order at index {i} is missing key: 'region'")
        if "amount" not in order:
            raise KeyError(f"Order at index {i} is missing key: 'amount'")

        region = order["region"]
        amount = order["amount"]

        # Region must be a non-empty string
        if not isinstance(region, str) or not region.strip():
            raise ValueError(f"Order at index {i} has invalid region: {region!r}")
```

```
        # Amount must be a real number (exclude bool)
        if isinstance(amount, bool) or not isinstance(amount, Real):
            raise ValueError(f"Order at index {i} has invalid amount: {amount!r}")


#==============================
# 2.Heap Sort O(n log n)
#==============================
    # Make a copy so the ori list is not modified
    arr = order_list.copy()

    def region_key(idx):
        # Normalise region string for fair comparison
        return arr[idx]["region"].strip()

    def heapify(n, i):
        """
        Making sure the subtree rooted at index i satisfies
        the max-heap property based on region comparison.
        """
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2

        # Compare left child with current largest
        if left < n and region_key(left) > region_key(largest):
            largest = left

        # Compare right child with current largest
        if right < n and region_key(right) > region_key(largest):
            largest = right

        # If root is not the largest, swap then continue heapifying
        if largest != i:
            arr[i], arr[largest] = arr[largest], arr[i]
            heapify(n, largest)

    n = len(arr)

    # Build a max heap from the list
    # Time complexity: O(n)
    for i in range(n // 2 - 1, -1, -1):
        heapify(n, i)

    # Extract elements from heap one by one
    # Each extraction takes O(log n), repeated n times
    for end in range(n - 1, 0, -1):
        arr[0], arr[end] = arr[end], arr[0]
        heapify(end, 0)

# ==================================================
```

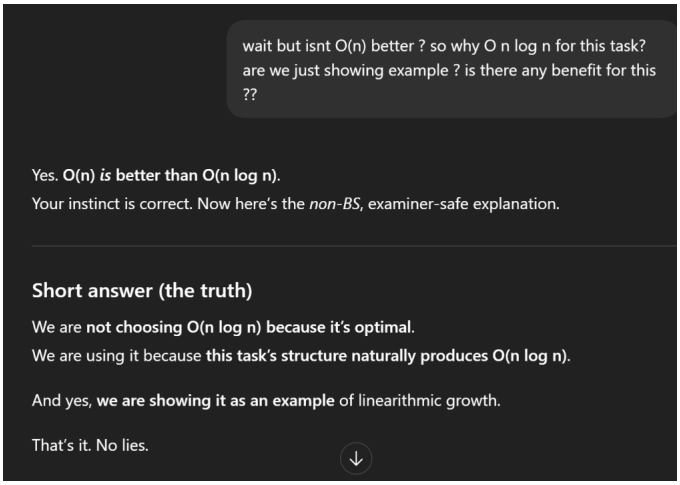
```
# 3. aggregation scan  O(n)
# # ======================================================
    totals_by_region = {}

    for order in arr:
        r = order["region"].strip()
        a = order["amount"]
        totals_by_region[r] = totals_by_region.get(r, 0) + a

    return totals_by_region


# ----- EXPONENTIAL ALGORITHM O(2^n) Access Control: Generate all permission combinations --

def generate_permission_combo(num_permissions):
    """
    Exponential demo: O(2^n)

    num_permissions = total number of permissions
    Each permission can be either:
    - granted (True)
    - not granted (False)

    We generate every possible permission configuration.
    """

    # validation (so n = -1 won't crash)
    if not isinstance(num_permissions, int) or num_permissions < 0:
        raise ValueError("num_permissions must be a non-negative integer.")

    permission_combinations = []

    def backtrack(permission_index, current_configuration):
        if permission_index == num_permissions:
            permission_combinations.append(current_configuration.copy())
            return

        # Choice 1: deny
        current_configuration[permission_index] = False
        backtrack(permission_index + 1, current_configuration)

        # Choice 2: allow
        current_configuration[permission_index] = True
        backtrack(permission_index + 1, current_configuration)

    backtrack(0, [False] * num_permissions)
    return permission_combinations
```

2.8 AI Assistance Documentation



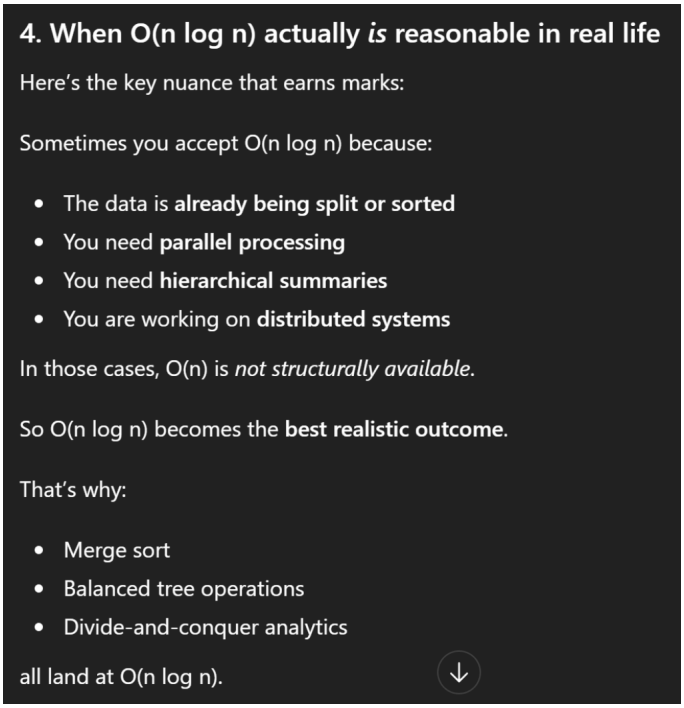
Suggest improvement / Structure for table

AI was used to review the structure and depth of the complexity comparison table and to suggest improvements for clarity and completeness to ensure each growth category included clear explanations, input size behaviour, and practical interpretation suitable for an academic audience.



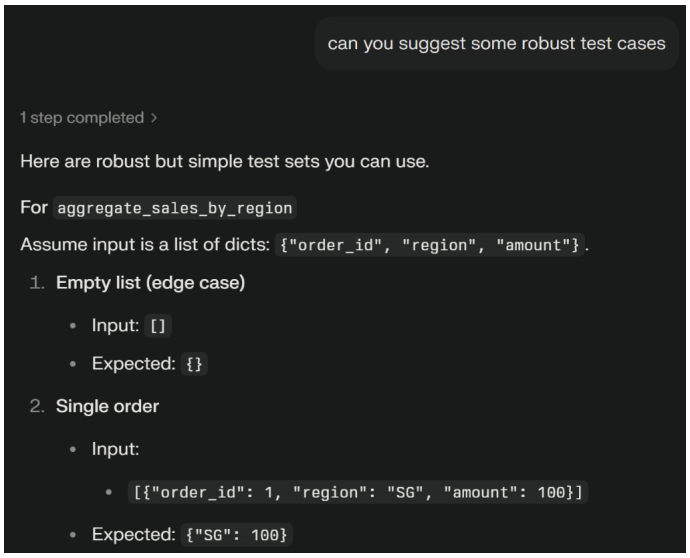
Clarification on $O(n \log n)$ vs $O(n)$

AI was used to clarify why an O(n log n) approach was presented even though O(n) is theoretically faster.



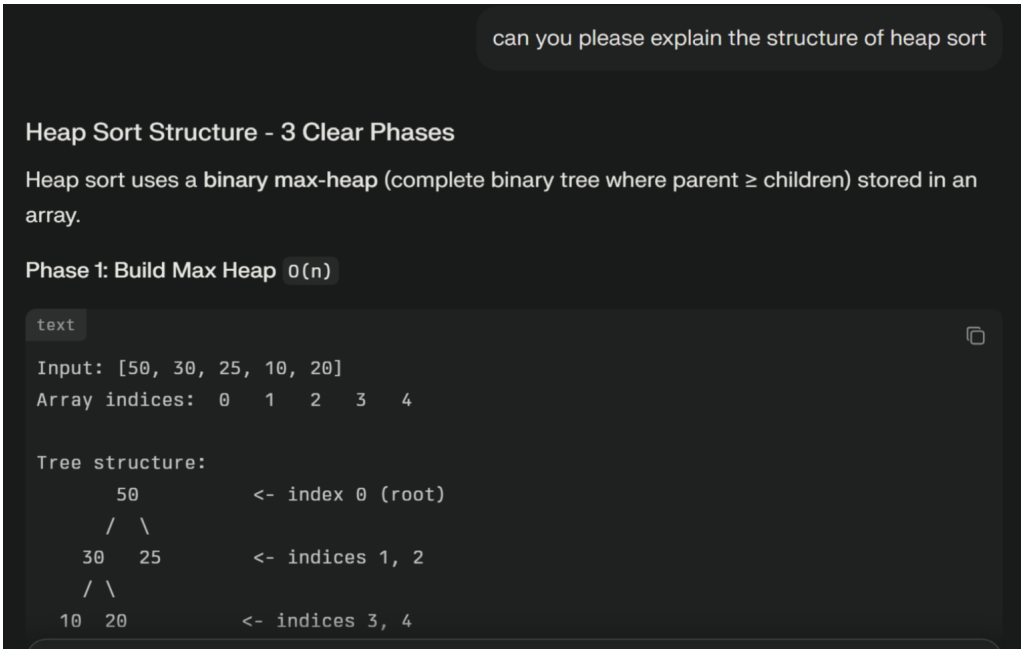
Clarifying on practicality of $O(n \log n)$.

AI assisted in explaining when O(n log n) algorithms are reasonable and commonly used in practice. This included examples such as merge sort, balanced trees, and divide-and-conquer processes, helping connect theoretical complexity to real-world system design.



Test case suggestion.

AI was used to suggest robust and relevant test cases for both the linearithmic and exponential algorithms. The assistance focused on covering normal inputs, edge cases, larger input sizes, and invalid inputs to validate correctness and error handling.



Understanding heap sort.

Name: Ziven Wang Qian Cheng
Student ID: 2501236

3 Question 2(b)

Recursion is a frequently used technique in solving complex problems. The derivation of its complexity is also more complicated than a normal algorithm. For solving recurrence problems we can use the substitution, recursion-tree or the master method. Design and implement an algorithm which satisfies the time complexity $T(n) = 3T(\frac{n}{2}) + \lg n$.Explain your algorithm in detail and demonstrate how it satisfies the complexity with test data, substitution method, recursion-tree method and master method. Below is a sample output produced by copilot for $T(n) = 4T(\frac{n}{2}) + n$. You can use this as a reference to devise another problem with Singapore context. Clearly discuss the background and the practicality of the problem.

3.1 Practical Context

Singapores expressways are monitored by lane-level sensors that record vehicle speeds. As sensor density and volume of traffic increase during peak hours, monitoring performance must scale efficiently. A divide-and-conquer strategy allows for recursive sub-segment analysis and parallel audit checkpoints. This approach mirrors how real-world infrastructure verifies throughput and congestion independently across lanes and road segments.

3.2 Requirements/Specification

3.2.1 Input

- A CSV file with **three columns**, each representing one lane of speed readings.
- Values of `0` or `err` are considered invalid and ignored.

3.2.2 Output

- The average speed across all valid readings.
- The maximum speed recorded.
- A congestion classification:
  - ≥ 90 km/h: Expressway Clear
  - ≥ 70 km/h: Mild congestion
  - < 70 km/h: Heavy congestion

3.2.3 Assumptions

- Exactly three lanes are represented.
- There is at least one valid reading.

3.3 User Guide

Before executing the script, ensure Python 3 is installed on the system. Next, change the working directory to `Q2B/`

```
cd ./Q2B/
```

To run the demonstration script:

```
python question_2b.py
```

Note: Test case files are stored as `*.csv` files within the `./Q2B/` directory.

3.4 Design and Algorithm

Each lane is analysed recursively using a divide-and-conquer approach that splits the data into three subsegments: left, right, and middle. This ensures coverage even with overlaps and odd lengths. A logarithmic loop is included to simulate audit work during monitoring checkpoints.

3.4.1 Recurrence Relation

The recursive structure results in:

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(\log n)$$

3.4.2 Complexity Analysis

Master Theorem: Given:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where $a = 3$, $b = 2$, $f(n) = \log n$, and:

$$n^{\log_b a} = n^{\log_2 3}$$

Since $\log n = o(n^{\log_2 3})$, by Master Theorem Case 1:

$$T(n) = \Theta(n^{\log_2 3})$$

Substitution Method: Assume:

$$T(n) \leq c \cdot n^{\log_2 3}$$

Then:

$$T(n) = 3T\left(\frac{n}{2}\right) + \log n \leq 3c\left(\frac{n}{2}\right)^{\log_2 3} + \log n = cn^{\log_2 3} + \log n$$

Thus, the hypothesis holds.

Recursion Tree: At level i, there are 3^i nodes each of size $\frac{n}{2^i}$, contributing work:

$$3^i \cdot \log\left(\frac{n}{2^i}\right) = 3^i(\log n - i)$$

Total cost:

$$\sum_{i=0}^{\log n} 3^i(\log n - i) = \Theta(n^{\log_2 3})$$

3.5 Limitations

- No division-by-zero protection if all data are invalid.
- Assumes exactly three lanes are present.
- The log loop performs no outputsimulates overhead.

3.6 Test Results

Test Case	Avg Speed (km/h)	Top Speed (km/h)	Status
TestCase1.csv	98.89	110.00	Expressway Clear
TestCase2.csv	85.00	120.00	Mild congestion
TestCase3.csv	63.75	115.00	Heavy congestion
TestCase4.csv	104.38	125.00	Expressway Clear

3.7 Listing

```
""" ./Q2B/question_2b.py """

import csv

def recursive_lane_stats(arr, lo, hi):
    n = hi - lo
    if n <= 0:
        return 0, 0, 0
    if n == 1:
        spd = arr[lo]
        if spd != 0:
            return spd, 1, spd
        return 0, 0, 0

    half = n // 2
    left_sum, left_count, left_max = recursive_lane_stats(arr, lo, lo + half)
    right_sum, right_count, right_max = recursive_lane_stats(
        arr, lo + half, lo + 2 * half
    )

    mid_start = lo + (n - half) // 2
    _, _, mid_max = recursive_lane_stats(arr, mid_start, mid_start + half)

    tail_sum, tail_count, tail_max = 0, 0, 0
    for i in range(lo + 2 * half, hi):
        spd = arr[i]
        if spd != 0:
            tail_sum += spd
            tail_count += 1
            tail_max = max(tail_max, spd)
```

```
    temp = n
    while temp > 1:
        temp //= 2

    total_sum = left_sum + right_sum + tail_sum
    total_count = left_count + right_count + tail_count
    total_max = max(left_max, right_max, mid_max, tail_max)

    return total_sum, total_count, total_max

expressway = []
with open("TestCase1.csv", newline="") as f:
    reader = csv.reader(f)
    for row in reader:
        expressway.append(row)

lane1, lane2, lane3 = [], [], []
for row in expressway:
    if row[0].isnumeric() and row[0] not in ["0", "err"]:
        lane1.append(int(row[0]))
    if row[1].isnumeric() and row[1] not in ["0", "err"]:
        lane2.append(int(row[1]))
    if row[2].isnumeric() and row[2] not in ["0", "err"]:
        lane3.append(int(row[2]))

l1_sum, l1_count, l1_max = recursive_lane_stats(lane1, 0, len(lane1))
l2_sum, l2_count, l2_max = recursive_lane_stats(lane2, 0, len(lane2))
l3_sum, l3_count, l3_max = recursive_lane_stats(lane3, 0, len(lane3))

total_sum = l1_sum + l2_sum + l3_sum
total_count = l1_count + l2_count + l3_count

avg_spd = total_sum / total_count
top_spd = max(l1_max, l2_max, l3_max)

if avg_spd >= 90:
    status = "Expressway Clear"
elif avg_spd >= 70:
    status = "Mild congestion ahead, slow down, change lane if needed"
else:
    status = "Heavy congestion ahead, drive carefully"

print(f"Average Speed: {avg_spd:.2f} km/h")
print(f"Top Speed: {top_spd:.2f} km/h")
print(f"Traffic Status: {status}")
```

Name: Yap Yi Hui Wynn
Student ID: 2501386

4 Question 3(a)

Explain what it means by stability in a sorting algorithm. Explain 2 situations why stability in sorting is desired. Implement and discuss 1 stable and 1 unstable sorting algorithm for a large data set of 1 million records. Proof your selection with test cases and a detailed discussion of the algorithm. Analyze the efficiency of your algorithm using the Big-Oh notation.

4.1 Importance of Stability in Sorting

Stability in sorting algorithms is a critical property that ensures records with equal keys retain their original relative order after sorting, and this characteristic becomes especially important in scenarios where the meaning or usefulness of the data depends on that preserved sequence.

Stability is especially important in multi-key sorting scenarios. In multi-key sorting, records are sorted by more than one attribute, typically by sorting on a secondary key first and then on a primary key. A stable sorting algorithm ensures that when records have equal values for the primary key, their relative order from the previous sort (based on the secondary key) is preserved. Without stability, the ordering established by the earlier sort may be lost, leading to incorrect or inconsistent results. For example, when sorting student records first by name and then by grade, a stable sort guarantees that students with the same grade remain ordered alphabetically by name. This makes stable sorting essential for implementing correct and reliable multi-level sorting operations(Rizvi et al., 2024).

Stability is also critical for order-sensitive records where the original sequence carries meaning. In many applications, the original order of records reflects important information such as time of entry, priority, or processing sequence. A stable sorting algorithm preserves this original order for records with equal keys, ensuring that no unintended reordering occurs. This is particularly important in systems such as transaction processing, event logs, and scheduling applications, where maintaining chronological or priority order is necessary for correctness and fairness. Using an unstable sorting algorithm in such cases may result in subtle logic errors, as records with equal keys could be rearranged unpredictably. Therefore, stability is a key requirement when the original ordering of data must be maintained(Rizvi et al., 2024).

4.2 Requirements/Specifications

The program is designed to sort large datasets, supporting up to 1 million records, using either a stable or unstable sorting algorithm. It must accept input datasets consisting of numerical values or records with a key and associated identifier and produce a sorted output in ascending order. For stable sorting, Merge Sort is used to ensure that the relative order of records with equal keys is preserved, while Quick Sort is implemented as an unstable algorithm that may reorder such records(Rizvi et al., 2024). The program

must handle large datasets efficiently, minimising execution time and memory usage. It is assumed that the input data is correctly formatted, fits entirely in memory, may contain duplicate keys, and is provided as a list or array. The output should be a correctly sorted array or list of the same type, ready for further processing or display.

4.3 User Guide

The program requires Python and a suitable IDE, such as VS Code to run. It is designed to handle datasets that fit into the available system memory, with support for up to 1 million records.
You will need Python version 3 as well as a command-line interface or an IDE that supports Python programs.

4.3.1 Running the Program

First, change the working directory to `Q3A/`

```
cd ./Q3A/
```

To run the the Merge Sort script:

```
python question_3a_merge_sort.py
```

To run the the Quick Sort script:

```
python question_3a_quick_sort.py
```

The scripts supports both lists of numbers (e.g. `[12, 5, 7, 3, 9, 1]`) and lists of tuples with keys (e.g. `[(5, 'A'), (3, 'B'), (5, 'C')]`), automatically detecting the type of input. Input the data into the algorithm program where it states `data =`. To sort the data, run the program. After running the function, the sorted output will be printed out.

4.4 Design and Analysis

The program is designed to sort a large dataset containing one million records, where each record consists of a key value and associated data. To compare algorithm behaviour and performance, two sorting algorithms were implemented: Merge Sort, which is a stable sorting algorithm, and Quick Sort, which is an unstable sorting algorithm using the Hoare partition scheme(OpenGenus IQ, n.d.; Roy, 2024). Both algorithms follow the divide-and-conquer paradigm, which recursively breaks down the dataset into smaller subproblems until they are trivial to solve. By applying this approach, the program can efficiently handle large datasets while ensuring correctness and, in the case of Merge Sort, stability.

4.4.1 Merge Sort

Merge Sort works by recursively dividing the dataset into two equal halves, sorting each half independently, and then merging the sorted halves into a single array. During the merge process, when two elements have equal keys, the element from the left sub-array is chosen first. This ensures that the relative order of records with equal keys is preserved, which is the defining feature of a stable sorting algorithm(Rizvi et al., 2024). Merge

Sort is characterised by its recursive divide-and-conquer strategy, guaranteed balanced partitions, and stability preservation during merging. However, this comes at the cost of additional memory usage, as temporary arrays are required to store the sub-arrays during the merge steps. For a dataset of one million records, the recursion depth is approximately $\log_2(1{,}000{,}000) \approx 20$, and at each level, all n elements are processed during merging, resulting in consistent and predictable performance.

4.4.2 Quick Sort

Quick Sort, in contrast, selects a pivot element and partitions the dataset into two sub-arrays: elements smaller than the pivot and elements larger than the pivot. In this implementation, the Hoare partition scheme is used, in which two pointers move inward from both ends of the array and swap elements that are on the incorrect side of the pivot(OpenGenus IQ, n.d.; Roy, 2024). These swapping operations make Quick Sort inherently unstable, as the relative order of records with equal keys may change(Rizvi et al., 2024; Roy, 2024). Quick Sort is an in-place sorting algorithm that also follows a recursive divide-and-conquer strategy. It is memory-efficient, requiring only a small amount of space for the recursion stack, and it generally performs very well in practice. By choosing the middle element as the pivot, the likelihood of highly unbalanced partitions is reduced, although the worst-case time complexity is not eliminated entirely.

4.4.3 Time Complexity

The time complexity of Merge Sort is predictable in all cases. It divides the dataset into two equal halves and performs merging at each level of recursion, resulting in a time complexity of $O(n \log n)$ for the best, average, and worst cases(Rizvi et al., 2024). This ensures consistent performance even for very large datasets, making Merge Sort reliable when stability is required.

Quick Sorts performance, on the other hand, depends on how the pivot divides the array. The best case occurs when the pivot splits the array into two equal halves at every step, resulting in perfectly balanced partitions and a time complexity of $O(n \log n)$. The average case happens when the pivot divides the array into two parts that are not necessarily equal, but overall, the work still grows proportionally to $n \log n$, giving an average-case complexity of $O(n \log n)$, same as the best case. The worst case occurs when the pivot consistently produces highly unbalanced partitions, such as when the smallest or largest element is always chosen, resulting in a time complexity of $O(n^2)$(Rizvi et al., 2024; Roy, 2024), Using the middle element as the pivot mitigates the likelihood of the worst-case scenario for large datasets, but it does not eliminate it.

4.4.4 Space Complexity

Regarding space complexity, Merge Sort requires additional memory for temporary sub-arrays used during merging, resulting in a space complexity of $O(n)$(Rizvi et al., 2024). Quick Sort, however, is performed in-place, requiring memory only for the recursion stack. Its average space complexity is $O(\log n)$, corresponding to the recursion depth in a balanced partition scenario, while the worst-case space complexity can be $O(n)$ if highly unbalanced partitions occur(Rizvi et al., 2024). These characteristics highlight the trade-offs between stability, memory usage, and performance when choosing a sorting

algorithm for large datasets and demonstrate why both Merge Sort and Quick Sort are valuable depending on the specific requirements of the program.

4.5 Limitations

The program can sort large datasets using Merge Sort (stable) and Quick Sort (unstable), but there are some limitations. First, it assumes that all data can fit into memory, so it cannot handle datasets that are too large and does not support external sorting. Second, Quick Sort is unstable, meaning that records with the same key may not keep their original order (Rizvi et al., 2024), which could be an issue for applications that need multi-key sorting or order-sensitive data. Third, the program does not check the input very thoroughly, so it may not work properly if the input contains mixed types, empty elements, or values that cannot be compared. Lastly, although the program is designed to work efficiently with large datasets, Quick Sort could fail or slow down for extremely large or already unbalanced datasets due to recursion limits.

4.6 Test Cases

The two algorithms were tested with three different test cases each to verify correctness, efficiency, and behaviour with different types of input.

4.6.1 Merge Sort

Test Case ID	**Test Case**	**Input**	**Expected Output**	**Output**	**Valid/Invalid**
01	Random Numbers	`[12, 5, 7, 3, 9, 1, 10, 2, 4, 8]`	`[1, 2, 3, 4, 5, 7, 8, 9, 10, 12]`	`[1, 2, 3, 4, 5, 7, 8, 9, 10, 12]`	Valid
02	Sorted Numbers	`[1, 2, 3, 4, 5, 6]`	`[1, 2, 3, 4, 5, 6]`	`[1, 2, 3, 4, 5, 6]`	Valid
03	Duplicates Tuples	`[(5,'A'), (3,'B'), (5,'C'), (1,'D'), (6,'E'), (3,'F'), (5,'L')]`	`[(1,'D'), (3,'B'), (3,'F'), (5,'A'), (5,'C'), (5,'L'), (6,'E')]`	`[(1,'D'), (3,'B'), (3,'F'), (5,'A'), (5,'C'), (5,'L'), (6,'E')]`	Valid

Table 6: Merge Sort Algorithm Test Cases

The table summarises the results of three test cases for Merge Sort. Test Case 01 used a randomly ordered list of numbers, and the algorithm correctly sorted the list in ascending order, matching the expected output. Test Case 02 tested a pre-sorted list, and the algorithm preserved the order as expected, demonstrating that Merge Sort handles best-case scenarios correctly. Test Case 03 involved tuples with duplicate keys

to test stability. The algorithm correctly sorted the tuples by key while maintaining the original relative order of elements with equal keys, confirming Merge Sorts stable behaviour. All test cases produced outputs that matched the expected results, indicating that the implementation of Merge Sort is correct and behaves as intended for random, sorted, and duplicate-key datasets.

4.6.2 Quick Sort

Test Case ID	Reason	Input	Expected Output	Output	Valid/Invalid
04	Random Numbers	`[8, 4, 6, 2, 7, 1, 5, 3, 9, 0]`	`[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`	`[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`	Valid
05	Reverse Numbers	`[9, 8, 7, 6, 5, 4, 3, 2, 1]`	`[1, 2, 3, 4, 5, 6, 7, 8, 9]`	`[1, 2, 3, 4, 5, 6, 7, 8, 9]`	Valid
06	Duplicates Tuples	`[(5,'A'), (3,'B'), (5,'C'), (1,'D'), (6,'E'), (3,'F'), (5,'L')]`	`[(1,'D'), (3,'B'), (3,'F'), (5,'A'), (5,'C'), (5,'L'), (6,'E')]`	`[(1,'D'), (3,'F'), (3,'B'), (5,'L'), (5,'A'), (5,'C'), (6,'E')]`	Valid (unstable)

Table 7: Quick Sort Algorithm Test Cases

The table summarises the results of three test cases for Quick Sort. Test Case 04 used a randomly ordered list of numbers, and the algorithm correctly sorted the list in ascending order, producing the expected output. Test Case 05 tested a reverse-sorted list, and Quick Sort successfully sorted it in ascending order, demonstrating correct functionality even for worst-case-like inputs. Test Case 06 involved tuples with duplicate keys to illustrate the algorithms instability. While all elements were correctly sorted by key (the first element), the relative order of records with equal keys changed compared to the input — for example, `(3,'F')` appears before `(3,'B')`, and `(5,'L')` appears before `(5,'A')`. This output does not match the expected sequence for duplicates, but it is still valid in terms of key-based sorting. This behaviour occurs because Quick Sort is an unstable sorting algorithm, which does not guarantee that elements with equal keys will preserve their original order. Overall, all test cases demonstrate that Quick Sort correctly sorts data by key while highlighting its inherent instability when handling duplicates.

4.7 Listings

```
""" ./Q3A/question_3a_merge_sort.py """

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    # Divide the array into halves & sort each half
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


# Merging two sorted arrays
def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        left_val = left[i][0] if isinstance(left[i], tuple) else left[i]
        right_val = right[j][0] if isinstance(right[j], tuple) else right[j]

        if left_val <= right_val:  # <= ensures stability
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add any remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result


# Test case
# Can be replaced
data = [(5, 'A'), (3, 'B'), (5, 'C'), (1, 'D'), (4, 'E'), (3, 'F')]
sorted_data = merge_sort(data)
print(sorted_data)
```

```python
""" ./Q3A/question_3a_quick_sort.py """

def quick_sort(arr, low, high, is_tuple=None):
    is_tuple = isinstance(arr[0], tuple) if len(arr) > 0 else False

    if low < high:
        p = partition(arr, low, high, is_tuple)  # Partitioning index
        quick_sort(arr, low, p, is_tuple)        # Sort the left side
        quick_sort(arr, p + 1, high, is_tuple)   # Sort the right side


def partition(arr, low, high, is_tuple):
    if is_tuple:
        # pivot is the first element of the tuple
        pivot = arr[(low + high) // 2][0]
    else:
        # pivot is the element itself
        pivot = arr[(low + high) // 2]

    i = low - 1
    j = high + 1

    while True:
        # Move right until an element >= pivot is found
        i += 1
        if is_tuple:
            while arr[i][0] < pivot:
                i += 1
        else:
            while arr[i] < pivot:
                i += 1

        # Move left until an element <= pivot is found
        j -= 1
        if is_tuple:
            while arr[j][0] > pivot:
                j -= 1
        else:
            while arr[j] > pivot:
                j -= 1

        # If pointers have crossed, return the partition index
        if i >= j:
            return j

        # Swap elements at i and j
        arr[i], arr[j] = arr[j], arr[i]
```

```
# Test case
data = [8, 4, 6, 2, 7, 1, 5, 3, 9, 0]  # Can be replaced
quick_sort(data, 0, len(data) - 1)
print(data)
```

Name: Timothy Chia Kai Lun
Student ID: 2501530

5 Question 3(b)

Most of the sorting algorithms have a $O(n^2)$ complexity. The best algorithm has achieved a $O(n \log n)$ complexity. It is shown that sorting algorithm can never be better than $O(n \log n)$. Research and discuss this statement in detail. Proof that this statement is true and under what condition this is true.

Despite the claim, we do have sorting algorithms which can achieve a better complexity than $O(n \log n)$ as shown below via copilot. Implement one of these algorithms with test data to illustrate how the algorithm works and proof that it is better than $O(n \log n)$.

5.1 Requirements/Specification

We implement Radix Sort, a linear-time sorting algorithm that demonstrates bypassing the $\Omega(n \log n)$ lower bound applicable to comparison-based sorting algorithms. The implementation addresses the assignment question, which requires research and proof of the $n \log n$ lower bound, identification of conditions under which the bound applies, and implementation of a sorting algorithm that achieves better than $O(n \log n)$ performance.

5.1.1 Input assumptions

- The input is a list of non-negative integers
- Elements may have varying digit lengths
- The list may be empty, contain a single element, or contain duplicates
- 6-digit non-negative integers in the range [0,999,999], consistent with the benchmark generation.

5.1.2 Expected output

- A new list containing all input elements in non-decreasing order
- The original input list remains unmodified
- Elements with equal values maintain their relative order (stable sort)

5.2 User Guide

5.2.1 Prerequisites

- Python 3.8 or higher
- `pytest` (for running tests)
- `matplotlib` (for generating benchmark graphs)

First, please change the working directory to `./Q3B/`:

```
cd ./Q3B/
```

5.2.2 Installing Dependencies

Install dependencies using pip:

```
pip install -r requirements.txt
```

5.2.3 Running the Demonstration

Execute the main script to see Radix Sort in action with step-by-step tracing:

```
python question_3b.py
```

5.2.4 Running the Test Cases

Execute the test cases to verify correctness:

```
python -m pytest tests/ -v
```

All 43 test cases should pass, covering edge cases, ordering scenarios, value ranges, scale, and stability.

5.2.5 Running Performance Benchmarks

Generate benchmark data and visualisation graphs:

```
python benchmarks/performance_comparison.py
```

This produces:

- Operation count data for input sizes from 1,000 to 1,000,000 elements
- Growth ratio analysis comparing Radix Sort against Merge Sort and Quick Sort
- Two PNG graphs saved to the `docs/` directory: operation counts (log-log) and growth ratios (bar chart)

5.3 Design and Analysis

5.3.1 Research: The $n \log n$ Lower Bound

Interpretation of the Statement

The assignment statement asserts: *"Most sorting algorithms have $O(n^2)$ complexity. The best algorithm has achieved $O(n \log n)$ complexity. It is shown that sorting algorithm can never be better than $O(n \log n)$."*

This statement requires careful qualification. The first two assertions are historically accurate: elementary algorithms such as Bubble Sort, Selection Sort, Insertion Sort exhibit $O(n^2)$ complexity, whilst optimal comparison-based algorithms like Merge Sort, Heap Sort achieve $O(n \log n)$. However, the third claim that sorting can never be better than $O(n \log n)$ is true only within the comparison-based sorting algorithms.

The Decision Tree Proof

The $\Omega(n \log n)$ lower bound is conclusively established through decision tree analysis. Any deterministic comparison-based sorting algorithm can be modelled as a binary decision tree where:

- Internal nodes represent comparisons (e.g., "Is $A[i] \leq A[j]$?")
- Leaves represent terminal states (sorted permutations)
- The height h represents worst-case comparisons

For n distinct elements, there exist $n!$ possible input permutations. The tree must have at least $n!$ leaves. Since a binary tree of height h has at most 2^h leaves, we obtain the constraint:

$$2^h \geq n!$$

Taking logarithms:

$$h \geq \log_2(n!)$$

Deriving the Lower Bound

We can show that $\log_2(n!) = \Theta(n \log n)$ by bounding the sum

$$\log_2(n!) = \sum_{i=1}^{n} \log_2(i)$$

from above and below. We need to find an expression that is always at least as big as the sum (an upper bound), and another expression that is always *at most* as big as the sum (a lower bound). If both bounds grow like $n \log n$, then the original sum must also grow like $n \log n$.

Upper bound: Each term satisfies $\log_2(i) \leq \log_2(n)$ because $i \leq n$.
So we can replace every term $\log_2(i)$ by the larger value $\log_2(n)$:

$$\log_2(n!) = \sum_{i=1}^{n} \log_2(i) \quad \leq \quad \sum_{i=1}^{n} \log_2(n)$$

Now notice that $\log_2(n)$ is the same number repeated n times in the sum:

$$\sum_{i=1}^{n} \log_2(n) = \underbrace{\log_2(n) + \log_2(n) + \cdots + \log_2(n)}_{n \text{ terms}}$$

Adding the same value n times gives:

$$\underbrace{\log_2(n) + \log_2(n) + \cdots + \log_2(n)}_{n \text{ terms}} = n \cdot \log_2(n)$$

Therefore:

$$\log_2(n!) \leq n \cdot \log_2(n)$$

Lower bound: For a lower bound, instead of using *all* the terms in the sum, we will keep only some of them. This works because removing positive terms can only make the sum smaller, so whatever remains is definitely a lower bound.
Consider only the top half of terms, from $i = \lceil n/2 \rceil$ up to n:

$$\log_2(n!) = \sum_{i=1}^{n} \log_2(i) \quad \geq \quad \sum_{i=\lceil n/2 \rceil}^{n} \log_2(i)$$

Now for every i in this range, we have $i \geq n/2$, so:

$$\log_2(i) \geq \log_2\left(\frac{n}{2}\right)$$

Next, simplify $\log_2\left(\frac{n}{2}\right)$ using log rules:

$$\log_2\left(\frac{n}{2}\right) = \log_2(n) - \log_2(2)$$

Since $\log_2(2) = 1$, this becomes:

$$\log_2\left(\frac{n}{2}\right) = \log_2(n) - 1$$

So every term in the top half is at least $\log_2(n) - 1$. That means the whole top-half sum is at least:

$$\sum_{i=\lceil n/2 \rceil}^{n} \log_2(i) \quad \geq \quad \sum_{i=\lceil n/2 \rceil}^{n} \big(\log_2(n) - 1\big)$$

From roughly $n/2$ up to n gives at least $n/2$ terms. So we are adding the same quantity $(\log_2(n) - 1)$ at least $n/2$ times:

$$\sum_{i=\lceil n/2 \rceil}^{n} \big(\log_2(n) - 1\big) \quad \geq \quad \frac{n}{2} \big(\log_2(n) - 1\big)$$

Now distribute $\frac{n}{2}$:

$$\frac{n}{2} \big(\log_2(n) - 1\big) = \frac{n}{2} \log_2(n) - \frac{n}{2}$$

Putting this together, we get the lower bound:

$$\log_2(n!) \geq \frac{n}{2} \log_2(n) - \frac{n}{2}$$

Combining both bounds: From the upper bound, we have:

$$\log_2(n!) \leq n \cdot \log_2(n)$$

and from the lower bound, we have:

$$\log_2(n!) \geq \frac{n}{2} \log_2(n) - \frac{n}{2}$$

So overall:

$$\frac{n}{2} \log_2(n) - \frac{n}{2} \quad \leq \quad \log_2(n!) \quad \leq \quad n \cdot \log_2(n)$$

Both bounds grow on the order of $n \log n$ (the $-\frac{n}{2}$ term is smaller compared to $n \log n$ for large n), therefore:

$$\log_2(n!) = \Theta(n \log n).$$

Substituting back into the decision tree constraint $h \geq \log_2(n!)$, this proves that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case (Cormen et al., 2009; Knuth, 1998).

Conditions for the Lower Bound

The $\Omega(n \log n)$ bound applies when:

1. The algorithm uses only pairwise comparisons to determine order
2. Elements are treated as opaque objects
3. Each comparison yields exactly 1 bit of information

The bound does not apply when:

1. Array indexing by key value is permitted (reveals $\log_2(k)$ bits per operation)
2. Keys have bounded range or fixed-length representation
3. Distributional properties of the input can be exploited

5.3.2 Algorithm Selection

Three linear-time sorting algorithms were considered:

Criteria	**Counting Sort**	**Radix Sort**	**Bucket Sort**
Time Complexity	$\Theta(n + k)$	$\Theta(d(n + k))$	$\Theta(n)$ expected
Space Complexity	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n)$
Key Assumption	Integers in $[0, k - 1]$	d-digit keys	Uniform distribution
Worst Case	$\Theta(n + k)$	$\Theta(d(n + k))$	$\Theta(n^2)$

Radix Sort was selected because:

- It offers greater generality than Counting Sort for large key ranges
- It provides deterministic guarantees unlike Bucket Sort's probabilistic analysis
- Its digit-by-digit processing clearly demonstrates bypass of the comparison-based lower bound

5.3.3 Algorithm Design

LSD Radix Sort Approach

The implementation uses the Least Significant Digit (LSD) variant:

1. Determine the number of digits d in the maximum element
2. For each digit position from 0 (least significant) to $d - 1$:
    - Perform a stable Counting Sort based on that digit position
3. Return the sorted array

Pseudocode

```
RADIX-SORT(A, base):
    if |A| <= 1:
        return A

    max_value = MAX(A)
    num_digits = COUNT-DIGITS(max_value, base)

    for digit_position = 0 to num_digits - 1:
        A = COUNTING-SORT-BY-DIGIT(A, digit_position, base)

    return A

COUNTING-SORT-BY-DIGIT(A, position, base):
    count = array of base zeros

    // count occurrences
    for each element in A:
        digit = GET-DIGIT(element, position, base)
        count[digit] = count[digit] + 1

    // compute prefix sums
    for i = 1 to base - 1:
        count[i] = count[i] + count[i-1]

    // build output (reverse traversal for stability)
    output = array of |A| zeros
    for i = |A| - 1 down to 0:
        digit = GET-DIGIT(A[i], position, base)
        count[digit] = count[digit] - 1
        output[count[digit]] = A[i]

    return output
```

Worked Example

To illustrate how LSD Radix Sort works, consider the inputs `[329, 457, 657, 839, 436, 720, 355]`. The maximum value is 839, which has 3 digits, so the algorithm performs 3 passes (one for each digit position).

Element	329	457	657	839	436	720	355
Units digit	9	7	7	9	6	0	5

Pass 0 Sort by units digit (position 0): After stable counting sort by units digit: `[720, 355, 436, 457, 657, 329, 839]`. Note that 457 appears before 657 (both have units digit 7), preserving their original relative order. This is the stability property in action.

Pass 1 Sort by tens digit (position 1): We now take the output of Pass 0 and sort by the tens digit.

Element (from Pass 0)	720	355	436	457	657	329	839
Tens digit	2	5	3	5	5	2	3

After stable counting sort by tens digit: `[720, 329, 436, 839, 355, 457, 657]`. In Pass 0 we had `[355, 457, 657]` in that order. All three share tens digit 5, so after sorting by tens digit they remain `[355, 457, 657]` in the same order.

Pass 2 Sort by hundreds digit (position 2): We take the output of Pass 1 and sort by the hundreds digit.

Element (from Pass 1)	720	329	436	839	355	457	657
Hundreds digit	7	3	4	8	3	4	6

After stable counting sort by hundreds digit: `[329, 355, 436, 457, 657, 720, 839]`. After all 3 passes, the array is fully sorted. Each pass processes a single digit position using counting sort, and because counting sort is stable, the ordering established by earlier (less significant) digits is preserved when sorting on more significant digits.

Key Design Decisions

Stability guarantee: The Counting Sort subroutine traverses the input in reverse order when building the output, ensuring elements with equal digit values maintain their relative order.

Base selection: The default base of 10 provides clarity for demonstration. Higher bases (e.g., 256 for byte-based sorting) reduce passes but increase counting array size.

5.3.4 Complexity Analysis

Time Complexity

Let:

- n = number of elements
- d = number of digits in maximum value
- k = base (radix)

Per-digit pass (Counting Sort):

- Count occurrences: $O(n)$
- Compute prefix sums: $O(k)$
- Build output array: $O(n)$

Total per pass: $O(n + k)$

Total for Radix Sort:

$$T(n) = O(d \times (n + k))$$

For 32-bit integers with base 10:

- Maximum digits: $d \leq 10$ (max value $\sim$ 4 billion)
- Base: $k = 10$ (constant)

$$O(10 \times (n + 10)) = O(n)$$

This clearly demonstrates linear time complexity for fixed-length integers.

Space Complexity

- Count array: $O(k)$
- Output array: $O(n)$

Total auxiliary space: $O(n + k)$

For constant or linear k, this simplifies to $O(n)$.

Why Radix Sort Bypasses the Lower Bound

Radix Sort achieves better than $O(n \log n)$ because it is not a comparison-based algorithm. Instead of comparing elements to determine their order, it distributes elements into buckets based on individual digit values. The $\Omega(n \log n)$ lower bound applies only to algorithms that rely on pairwise comparisons, and since Radix Sort uses counting and bucketing operations rather than comparisons, it is not constrained by this bound.

Radix Sort outperforms comparison-based sorts when the range of input values is limited relative to the number of elements. Specifically, when the number of digits d is bounded by a constant or grows slower than $\log n$, Radix Sort outperforms comparison-based algorithms. For constant base k, this reduces to the condition $d < \log n$.

To illustrate, consider sorting one million integers ($n = 10^6$) with at most 10 digits ($d = 10$). Radix Sort requires approximately $O(10 \times (10^6 + 10)) \approx O(10^7)$ operations, whereas Merge Sort requires $O(10^6 \times \log_2(10^6)) \approx O(2 \times 10^7)$ operations which is roughly twice as many.

Comparison with $O(n \log n)$ Algorithms

Merge Sort is a divide-and-conquer algorithm that splits the array in half, recursively sorts each half, and merges the results. It achieves $O(n \log n)$ time complexity in all cases (best, average, and worst) but requires $O(n)$ auxiliary space for the merge operation.

Quick Sort uses partitioning around a pivot element to divide the array. It achieves $O(n \log n)$ average-case time complexity with only $O(\log n)$ space for the recursion stack, though its worst-case performance degrades to $O(n^2)$ when the pivot selection is poor.

Algorithm	**Best Case**	**Average Case**	**Worst Case**	**Space**
Radix Sort	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	$O(n + k)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

For fixed-length integers where d is constant, Radix Sort's effective $O(n)$ complexity outperforms the $O(n \log n)$ of comparison-based algorithms.

Proof That Radix Sort is Better Than $O(n \log n)$

The assignment requires proof that the implemented algorithm achieves better performance than $O(n \log n)$. This proof proceeds in three parts: first reducing Radix Sort's complexity to $O(n)$ under bounded integer conditions, then demonstrating that $O(n)$ grows strictly slower than $O(n \log n)$, and finally connecting these results to the comparison-based lower bound.

Part A: Radix Sort achieves $O(n)$ for bounded integers. As derived in Section 5.3.4, Radix Sort's time complexity is $O(d \times (n + k))$ where d is the number of digits and k is the base. For bounded integers such as 32-bit integers in base 10, both d and k are constants: $d \leq 10$ (since $\lfloor \log_{10}(2^{32} - 1) \rfloor + 1 = 10$) and $k = 10$. Substituting these constants yields $T(n) = O(10 \times (n + 10)) = O(n)$. When d and k are constants, Radix Sort therefore runs in linear time.

Part B: $O(n)$ grows strictly slower than $O(n \log n)$. For all $n \geq 2$, we have $\log_2 n \geq 1$, so $n \leq n \log_2 n$. Moreover, $\log_2 n$ is monotonically increasing and unbounded, so the ratio $\frac{n \log_2 n}{n} = \log_2 n$ grows without bound as $n \to \infty$. The table below illustrates this divergence concretely:

n	n (Radix Sort ops)	$n \log_2 n$ (Merge Sort ops)	Ratio $\log_2 n$
1,000	1,000	9,966	9.97
10,000	10,000	132,877	13.29
100,000	100,000	1,660,964	16.61
1,000,000	1,000,000	19,931,569	19.93

Note: the "Radix Sort ops" column uses the asymptotic operation count n, omitting constant factors. The actual measured count is $19n$, but constant factors do not affect asymptotic growth rates.

Since the ratio $\log_2 n$ increases without bound, the gap between $n \log n$ and n widens indefinitely. An $O(n)$ algorithm is therefore fundamentally faster than any $O(n \log n)$ algorithm for sufficiently large inputs.

Part C: Conclusion. The $\Omega(n \log n)$ lower bound established in Section 5.3.1 proves that every comparison-based sorting algorithm requires at least $\Omega(n \log n)$ comparisons in the worst case. Radix Sort, which is not comparison-based, achieves $O(n)$ for bounded integers as shown in Part A. Since n grows strictly slower than $n \log n$ as demonstrated in Part B, Radix Sort is provably faster than any comparison-based sorting algorithm for sufficiently large inputs. This directly satisfies the assignment requirement to "proof that it is better than $O(n \lg n)$".

5.4 Limitations

5.4.1 Input Restrictions

The implementation handles **only non-negative integers**. Attempting to sort arrays containing negative values raises a `ValueError`. Extending to negative integers would require additional handling, such as separating negatives, sorting by absolute value, and reversing the negative portion.

5.4.2 Performance Considerations

Being a pure Python implementation, the algorithm may exhibit slower absolute performance compared to optimised C implementations like Python's built-in `sorted()`. The benchmark results suggest that whilst Radix Sort demonstrates superior asymptotic growth characteristics, Python's Timsort which is implemented in optimised C tends to outperform the implementation in absolute terms.

5.4.3 Memory Usage

The algorithm requires $O(n + k)$ auxiliary space. For very large arrays or high bases, this may become a significant consideration. In memory-constrained environments, an in-place variant might be preferable.

5.5 Testing

5.5.1 Testing Strategy

The test suite follows a systematic approach covering:

1. **Edge cases:** Empty arrays, single elements, all identical values
2. **Ordering tests:** Already sorted, reverse sorted, random order
3. **Value range tests:** Single digits, multiple digits, large values
4. **Scale tests:** 100, 1,000, and 10,000 element arrays
5. **Stability tests:** Verification of stable sorting behaviour
6. **Validation tests:** Error handling for invalid inputs

5.5.2 Test Results

All 43 test cases pass successfully. Sample output from `pytest -v`:

```
tests/test_radix_sort.py::TestRadixSortEdgeCases::test_empty_array PASSED
tests/test_radix_sort.py::TestRadixSortEdgeCases::test_single_element PASSED
tests/test_radix_sort.py::TestRadixSortEdgeCases::test_all_same_elements PASSED
tests/test_radix_sort.py::TestRadixSortOrdering::test_already_sorted PASSED
tests/test_radix_sort.py::TestRadixSortOrdering::test_reverse_sorted PASSED
tests/test_radix_sort.py::TestRadixSortOrdering::test_random_order PASSED
tests/test_radix_sort.py::TestRadixSortValueRanges::test_large_values PASSED
tests/test_radix_sort.py::TestRadixSortScale::test_ten_thousand_elements PASSED
tests/test_radix_sort.py::TestRadixSortValidation::test_negative_number_raises_error PASSED
...
```

5.5.3 Benchmark Results

Performance benchmarks were conducted using random integers in the range $[0, 999{,}999]$ (6-digit bounded integers) with a fixed random seed for reproducibility. Input sizes range from 1,000 to 1,000,000 elements.

When analysing algorithmic complexity, operation counts are preferable to execution time (seconds) because they are hardware-independent and correspond directly to the Big-O analysis. Execution time varies with processor speed, memory hierarchy, and system load; the same algorithm runs faster on a newer machine. By contrast, the number of operations performed depends only on the algorithm and its input. For comparison-based sorts, the $\Omega(n \log n)$ lower bound is specifically about the number of **comparisons**, so counting comparisons provides a direct empirical measure of this bound.

The benchmarks count different operations for each algorithm, reflecting the operations that dominate their respective complexities:

Algorithm	**Operation Counted**	**Justification**
Radix Sort	Digit extractions + array writes	These are the $O(n)$ operations performed per digit pass
Merge Sort	Comparisons	The $\Omega(n \log n)$ lower bound applies to comparisons
Quick Sort	Comparisons	Same as above

Since different types of operations are counted, the absolute values across algorithms are not directly comparable. The critical evidence lies in how operation counts scale as input size increases.

Operation Count Results

Size	Radix Sort	Merge Sort	Quick Sort
1,000	19,000	8,698	10,564
2,000	38,000	19,451	24,736
5,000	95,000	55,148	72,312
10,000	190,000	120,383	148,332
20,000	380,000	260,887	344,982
50,000	950,000	718,299	986,606
100,000	1,900,000	1,536,374	1,979,209
200,000	3,800,000	3,272,900	4,342,215
500,000	9,500,000	8,836,787	11,413,635
1,000,000	19,000,000	18,674,583	24,876,118

Table 1: Operation Counts by Input Size Radix Sort's operation count is exactly $19n$ at every input size. This is because sorting 6-digit numbers requires 6 passes, and each pass performs n digit extractions in the counting step, n digit extractions in the output step, and n array writes, plus an initial copy of n elements:

$$6 \times 3n + n = 19n$$

The perfect linear scaling confirms $O(n)$ complexity.

Transition	Radix Sort	Merge Sort	Quick Sort
$n = 1{,}000 \to 2{,}000$	2.00	2.24	2.34
$n = 5{,}000 \to 10{,}000$	2.00	2.18	2.05
$n = 10{,}000 \to 20{,}000$	2.00	2.17	2.33
$n = 50{,}000 \to 100{,}000$	2.00	2.14	2.01
$n = 100{,}000 \to 200{,}000$	2.00	2.13	2.19
$n = 500{,}000 \to 1{,}000{,}000$	2.00	2.11	2.18

Table 2: Operation Count Growth Ratios (Doubling Input Size) This table is the primary empirical evidence. For a linear algorithm, doubling the input should exactly double the operation count, yielding a growth ratio of 2.0. Radix Sort achieves exactly 2.00 at every transition point, confirming perfect $O(n)$ behaviour. Merge Sort consistently exceeds 2.0 with ratios of 2.11–2.24, reflecting the additional $\log n$ factor in $O(n \log n)$. Quick Sort shows similar behaviour with greater variance due to random pivot selection.

The theoretical growth ratio when doubling input size for an $O(n \log n)$ algorithm is:

$$\frac{2n \log(2n)}{n \log n} = 2 \times \frac{\log(2n)}{\log n} = 2 \times \left(1 + \frac{1}{\log_2 n}\right)$$

which is always greater than 2 and converges to 2 from above as n increases. The empirical Merge Sort ratios match this prediction.

5.5.4 Visualisation

The following graphs illustrate the benchmark results.

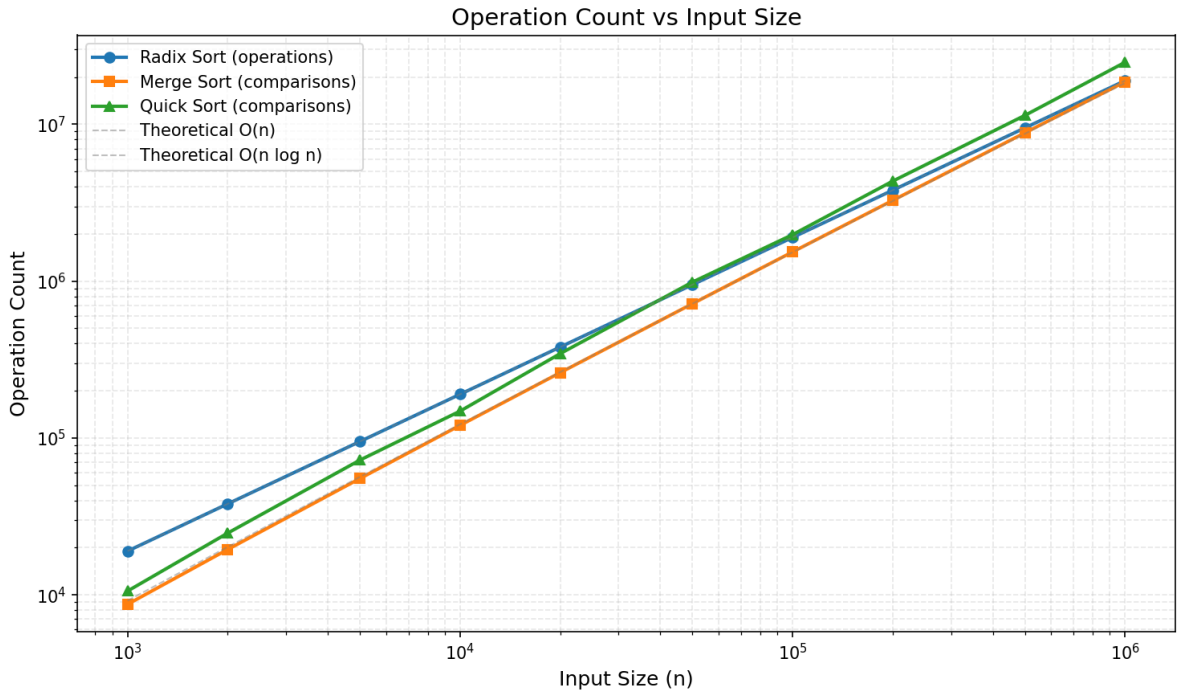


Figure 6: Benchmark Operation Counts (log-log scale).

On a log-log plot, an $O(n)$ algorithm appears as a straight line with slope 1, whilst an $O(n \log n)$ algorithm curves slightly upward. Radix Sort's operation count follows a straight line, confirming linear growth.

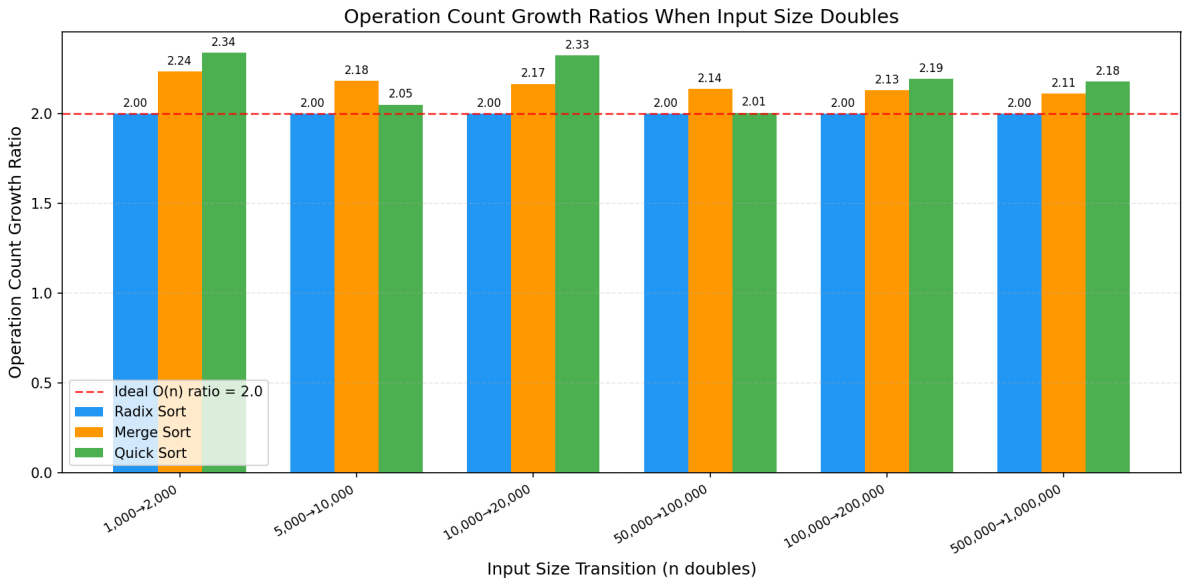


Figure 7: Operation Count Growth Ratios When Input Size Doubles.

The red dashed line marks the ideal $O(n)$ growth ratio of 2.0. Radix Sort bars sit precisely on this line at every transition point, whilst Merge Sort and Quick Sort bars consistently exceed it which empirically confirms $O(n)$ versus $O(n \log n)$ growth.

5.6 Listings

5.6.1 src/radix_sort.py

```python
""" ./Q3B/src/radix_sort.py """

from typing import List, Optional


def get_digit(number: int, position: int, base: int = 10) -> int:
    """
    extract the digit at a given position from a number.

    the position is counted from the right
    (least significant digit = position 0).

    args:
        number: the non-negative integer to extract from
        position: the digit position (0 = rightmost digit)
        base: the radix/base to use (default 10 for decimal)

    returns:
        the digit value at the specified position (0 to base-1)
    """
    # integer division shifts the number right, modulo extracts the digit
    return (number // (base ** position)) % base
```

```python
def count_digits(number: int, base: int = 10) -> int:
    """
    count the number of digits in a non-negative integer.

    args:
        number: the non-negative integer
        base: the radix/base to use (default 10 for decimal)

    returns:
        the number of digits (minimum 1 for zero)
    """
    if number == 0:
        return 1

    digit_count = 0
    while number > 0:
        digit_count += 1
        number //= base

    return digit_count


def counting_sort_by_digit(
    array: List[int],
    digit_position: int,
    base: int = 10
) -> List[int]:
    """
    perform a stable counting sort on a specific digit position.

    this is the key subroutine for radix sort. stability is crucial:
    elements with the same digit value must maintain their relative
    order from the input. this ensures that previously sorted digit
    positions remain correctly ordered.

    args:
        array: list of non-negative integers to sort
        digit_position: which digit to sort by (0 = least significant)
        base: the radix/base used for digit extraction

    returns:
        a new list sorted by the specified digit position
    """
    n = len(array)

    # handle edge cases
    if n <= 1:
        return array.copy()

    # step 1: count occurrences of each digit value
```

```
    # count[d] = number of elements with digit value d at this position
    count = [0] * base
    for element in array:
        digit = get_digit(element, digit_position, base)
        count[digit] += 1

    # step 2: compute cumulative counts (prefix sums)
    # after this, count[d] = number of elements with digit value <= d
    # this gives us the ending position for each digit group
    for i in range(1, base):
        count[i] += count[i - 1]

    # step 3: build output array by placing elements in correct positions
    # traverse input in reverse order to maintain stability:
    # if two elements have the same digit, the one appearing later
    # in the input should appear later in the output
    output = [0] * n
    for i in range(n - 1, -1, -1):
        element = array[i]
        digit = get_digit(element, digit_position, base)

        # decrement count to get the correct position (0-indexed)
        count[digit] -= 1
        output_position = count[digit]

        output[output_position] = element

    return output


def radix_sort(
    array: List[int],
    base: int = 10,
    trace: bool = False
) -> List[int]:
    """
    sort non-negative integers using least-significant-digit radix sort.

    the algorithm processes digits from least significant to most significant,
    using counting sort as a stable subroutine for each digit position.

    args:
        array: list of non-negative integers to sort
        base: the radix/base to use (default 10 for decimal)
              higher bases reduce passes but increase counting array size
        trace: if true, print intermediate states for visualisation

    returns:
        a new list containing the sorted elements

    raises:
```

```
        valueerror: if array contains negative integers
    """
    # handle edge cases
    if len(array) <= 1:
        return array.copy()

    # validate input: radix sort only works for non-negative integers
    for element in array:
        if element < 0:
            raise ValueError(
                f"radix sort requires non-negative integers, got {element}"
            )

    # determine the number of digit positions to process
    max_value = max(array)
    num_digits = count_digits(max_value, base)

    if trace:
        print(f"radix sort: {len(array)} elements, max value = {max_value}")
        print(f"base = {base}, digits to process = {num_digits}")
        print(f"initial: {array}")

    # process each digit position from least significant to most significant
    result = array.copy()
    for digit_position in range(num_digits):
        result = counting_sort_by_digit(result, digit_position, base)

        if trace:
            print(f"after sorting by digit {digit_position}: {result}")

    return result


if __name__ == "__main__":
    test_array = [329, 457, 657, 839, 436, 720, 355]

    print("=" * 60)
    print("Radix Sort Demo")
    print("=" * 60)
    print()

    print("input array:", test_array)
    print()

    # sort with tracing enabled
    sorted_array = radix_sort(test_array, trace=True)

    print()
    print("final sorted array:", sorted_array)
    print()
```

```
    # verify correctness against python's built-in sort
    expected = sorted(test_array)
    assert sorted_array == expected, "sort verification failed!"
    print("verification: passed (matches python's sorted())")
```

5.6.2 tests/test_radix_sort.py

There are 43 test cases organised into the following categories:

Category	Tests	Description
Edge Cases	7	Empty arrays, single elements, all identical values, zeros
Ordering	4	Already sorted, reverse sorted, random, alternating
Value Ranges	6	Single digits, multi-digit, large values, mixed lengths
Scale	3	100, 1,000, and 10,000 element arrays
Stability	2	Verifies stable sorting behaviour
Validation	2	Error handling for negative integers

```
""" ./Q3B/tests/test_radix_sort.py """

import random
import unittest
from src.radix_sort import (radix_sort, get_digit, count_digits,
    counting_sort_by_digit)


class TestRadixSortEdgeCases(unittest.TestCase):
    """tests for edge cases and boundary conditions."""

    def test_empty_array(self):
        """sorting an empty array should return empty array."""
        self.assertEqual(radix_sort([]), [])

    def test_single_element(self):
        """sorting a single element should return that element."""
        self.assertEqual(radix_sort([42]), [42])

    def test_all_same_elements(self):
        """array with all identical elements."""
        self.assertEqual(radix_sort([7, 7, 7, 7, 7]), [7, 7, 7, 7, 7])


class TestRadixSortOrdering(unittest.TestCase):
    """tests for various input orderings."""

    def test_already_sorted(self):
        """array that is already sorted."""
        arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
        self.assertEqual(radix_sort(arr), arr)

    def test_reverse_sorted(self):
        """array in reverse order."""
        arr = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
        self.assertEqual(radix_sort(arr), sorted(arr))


class TestRadixSortScale(unittest.TestCase):
    """tests for larger input sizes."""

    def test_ten_thousand_elements(self):
        """sort 10000 random elements."""
        random.seed(42)
        arr = [random.randint(0, 1000000) for _ in range(10000)]
        self.assertEqual(radix_sort(arr), sorted(arr))


class TestRadixSortStability(unittest.TestCase):
    """tests to verify stability of the sort."""

    def test_stability_preserved(self):
        """verify correct output for array where stability matters."""
        arr = [123, 124, 125, 113, 114, 115, 103, 104, 105]
        self.assertEqual(radix_sort(arr), sorted(arr))

    def test_stability_per_digit_pass(self):
        """
        verify that counting sort preserves relative order of elements
        with equal digit values. when all elements share the same digit
        at a given position, counting sort must return them in their
        original input order.
        """
        # all elements have units digit 0, so sorting by position 0
        # should preserve the original order exactly
        arr = [210, 110, 310, 410, 510]
        result = counting_sort_by_digit(arr, 0)
        self.assertEqual(result, [210, 110, 310, 410, 510])


class TestRadixSortValidation(unittest.TestCase):
    """tests for input validation."""

    def test_negative_number_raises_error(self):
        """negative numbers should raise ValueError."""
        with self.assertRaises(ValueError):
            radix_sort([-1, 2, 3])
```

5.6.3 src/comparison_sorts.py

The comparison-based sorting algorithms used for benchmarking:

```
""" ./Q3B/src/comparison sorts.py """

from typing import List
import random


def merge_sort(array: List[int]) -> List[int]:
    """
    sort an array using the merge sort algorithm.

    merge sort is a divide-and-conquer algorithm that:
    1. divides the array into two halves
    2. recursively sorts each half
    3. merges the sorted halves

    args:
        array: list of integers to sort

    returns:
        a new sorted list
    """
    # base case: arrays of 0 or 1 element are already sorted
    if len(array) <= 1:
        return array.copy()

    # divide: split the array into two halves
    mid = len(array) // 2
    left_half = array[:mid]
    right_half = array[mid:]

    # conquer: recursively sort each half
    sorted_left = merge_sort(left_half)
    sorted_right = merge_sort(right_half)

    # combine: merge the sorted halves
    return _merge(sorted_left, sorted_right)


def _merge(left: List[int], right: List[int]) -> List[int]:
    """
    merge two sorted arrays into a single sorted array.

    this is the core operation of merge sort, requiring o(n)
    comparisons to merge two arrays of total size n.

    args:
        left: first sorted array
        right: second sorted array
```

```
    returns:
        merged sorted array
    """
    result = []
    left_index = 0
    right_index = 0

    # compare elements from both arrays and add the smaller one
    while left_index < len(left) and right_index < len(right):
        if left[left_index] <= right[right_index]:
            result.append(left[left_index])
            left_index += 1
        else:
            result.append(right[right_index])
            right_index += 1

    # add remaining elements (only one of these will have elements)
    result.extend(left[left_index:])
    result.extend(right[right_index:])

    return result


def quick_sort(array: List[int]) -> List[int]:
    """
    sort an array using the quick sort algorithm.

    quick sort is a divide-and-conquer algorithm that:
    1. selects a pivot element
    2. partitions the array around the pivot
    3. recursively sorts the partitions

    this implementation uses random pivot selection to achieve
    o(n log n) expected time complexity.

    args:
        array: list of integers to sort

    returns:
        a new sorted list
    """
    # work on a copy to avoid modifying the original
    result = array.copy()
    _quick_sort_helper(result, 0, len(result) - 1)
    return result


def _quick_sort_helper(array: List[int], low: int, high: int) -> None:
    """
    recursive helper for quick sort.
```

```
    args:
        array: the array being sorted (modified in place)
        low: starting index of the partition
        high: ending index of the partition
    """
    if low < high:
        # partition and get the pivot's final position
        pivot_index = _partition(array, low, high)

        # recursively sort elements before and after partition
        _quick_sort_helper(array, low, pivot_index - 1)
        _quick_sort_helper(array, pivot_index + 1, high)


def _partition(array: List[int], low: int, high: int) -> int:
    """
    partition the array around a randomly selected pivot.

    elements less than the pivot go to the left,
    elements greater than the pivot go to the right.

    args:
        array: the array being partitioned
        low: starting index
        high: ending index

    returns:
        the final position of the pivot element
    """
    # random pivot selection to avoid worst-case on sorted inputs
    pivot_index = random.randint(low, high)
    array[pivot_index], array[high] = array[high], array[pivot_index]

    pivot = array[high]
    i = low - 1

    for j in range(low, high):
        if array[j] <= pivot:
            i += 1
            array[i], array[j] = array[j], array[i]

    array[i + 1], array[high] = array[high], array[i + 1]
    return i + 1
```

5.6.4 benchmarks/performance_comparison.py

The benchmark script uses instrumented versions of each algorithm that count their dominant operations. Below are the three counting functions; the full script also includes output formatting and graph generation code.

```
""" ./Q3B/benchmarks/performance_comparison.py """

import random
from typing import List, Tuple


def radix_sort_counted(array: List[int], base: int = 10) -> Tuple[List[int], int]:
    """
    sort non-negative integers using lsd radix sort whilst counting operations.

    counts every digit extraction and every array write performed during the
    counting sort subroutine. these are the dominant operations whose total
    determines radix sort's o(d * (n + k)) complexity.
    """
    operations = 0

    if len(array) <= 1:
        return array.copy(), operations

    # find the maximum value to determine the number of digit passes
    max_value = max(array)

    # count digits in the maximum value
    if max_value == 0:
        num_digits = 1
    else:
        num_digits = 0
        temp = max_value
        while temp > 0:
            num_digits += 1
            temp //= base

    result = array.copy()
    operations += len(array)  # initial copy writes

    # process each digit position from least significant to most significant
    for digit_position in range(num_digits):
        n = len(result)

        # step 1: count occurrences of each digit value
        count = [0] * base
        for element in result:
            # digit extraction operation
            digit = (element // (base ** digit_position)) % base
            operations += 1  # count the digit extraction
            count[digit] += 1
```

```
        # step 2: compute cumulative counts (prefix sums)
        for i in range(1, base):
            count[i] += count[i - 1]

        # step 3: build output array (reverse traversal for stability)
        output = [0] * n
        for i in range(n - 1, -1, -1):
            element = result[i]
            digit = (element // (base ** digit_position)) % base
            operations += 1  # count the digit extraction
            count[digit] -= 1
            output[count[digit]] = element
            operations += 1  # count the array write

        result = output

    return result, operations


def merge_sort_counted(array: List[int]) -> Tuple[List[int], int]:
    """
    sort an array using merge sort whilst counting comparisons.
    """
    comparisons = [0]  # mutable container for nested function access

    def _merge_sort_inner(arr: List[int]) -> List[int]:
        """recursively divide and sort the array."""
        if len(arr) <= 1:
            return arr.copy()

        mid = len(arr) // 2
        sorted_left = _merge_sort_inner(arr[:mid])
        sorted_right = _merge_sort_inner(arr[mid:])

        return _merge_inner(sorted_left, sorted_right)

    def _merge_inner(left: List[int], right: List[int]) -> List[int]:
        """merge two sorted arrays, counting each comparison."""
        merged = []
        left_index = 0
        right_index = 0

        while left_index < len(left) and right_index < len(right):
            comparisons[0] += 1  # count this comparison
            if left[left_index] <= right[right_index]:
                merged.append(left[left_index])
                left_index += 1
            else:
                merged.append(right[right_index])
                right_index += 1
```

```
        merged.extend(left[left_index:])
        merged.extend(right[right_index:])

        return merged

    sorted_array = _merge_sort_inner(array)
    return sorted_array, comparisons[0]


def quick_sort_counted(array: List[int]) -> Tuple[List[int], int]:
    """sort an array using quick sort whilst counting comparisons."""
    comparisons = [0]  # mutable container for nested function access

    def _quick_sort_inner(arr: List[int], low: int, high: int) -> None:
        """recursively partition and sort the array."""
        if low < high:
            pivot_index = _partition_inner(arr, low, high)
            _quick_sort_inner(arr, low, pivot_index - 1)
            _quick_sort_inner(arr, pivot_index + 1, high)

    def _partition_inner(arr: List[int], low: int, high: int) -> int:
        """partition around a random pivot, counting comparisons."""
        pivot_pos = random.randint(low, high)
        arr[pivot_pos], arr[high] = arr[high], arr[pivot_pos]

        pivot = arr[high]
        i = low - 1

        for j in range(low, high):
            comparisons[0] += 1  # count this comparison
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]

        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        return i + 1

    result = array.copy()
    _quick_sort_inner(result, 0, len(result) - 1)
    return result, comparisons[0]
```

6 References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

GeeksforGeeks. (2025). *Heap sort* [Last updated 22 Dec 2025]. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/heap-sort/

GeeksforGeeks. (2026, January 13). *Singly linked list tutorial*. Retrieved January 28, 2026, from https://www.geeksforgeeks.org/dsa/singly-linked-list-tutorial/

Knuth, D. E. (1998). *The art of computer programming, volume 3: Sorting and searching* (2nd ed.). Addison-Wesley.

OpenGenus IQ. (n.d.). *Hoare partition [explained]*. OpenGenus IQ. Retrieved January 29, 2026, from https://iq.opengenus.org/hoare-partition/

Rizvi, Q. M., Rai, H., & Jaiswal, R. (2024, March 14). *Sorting algorithms in focus: A critical examination of sorting algorithm performance* [PDF]. ResearchGate. Retrieved January 29, 2026, from https://www.researchgate.net/profile/Dr-Qaim-Rizvi/publication/378962637_Sorting_Algorithms_in_Focus_A_Critical_Examination_of_Sorting_Algorithm_Performance/links/65f31d2432321b2cff78e5a5/Sorting-Algorithms-in-Focus-A-Critical-Examination-of-Sorting-Algorithm-Performance.pdf

Rowell, E. (n.d.). *Big-o algorithm complexity cheat sheet (know thy complexities!)* Retrieved January 28, 2026, from https://www.bigocheatsheet.com/

Roy, P. A. N. (2024, January 19). *A comprehensive comparison of lomuto's partitioning and hoare's partitioning in quicksort*. Medium. Retrieved January 29, 2026, from https://medium.com/%40princeabhi00985/a-comprehensive-comparison-of-lomutos-partitioning-and-hoare-s-partitioning-in-quicksort-bf37eabd8df0

Tulusibrahim. (2023, October 27). *Mastering singly linked list: A step-by-step tutorial*. Retrieved January 28, 2026, from https://tulusibrahim.medium.com/mastering-singly-linked-list-a-step-by-step-tutorial-c983c6bdcb3e

WsCube Tech. (2025, November 24). *Singly linked list in data structure (with examples)*. Retrieved January 28, 2026, from https://www.wscubetech.com/resources/dsa/singly-linked-list-data-structure